

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук _____

Кафедра _____ програмної інженерії _____

КОМПЛЕКСНИЙ КУРСОВИЙ ПРОЄКТ
Пояснювальна записка

рівень вищої освіти _____ перший (бакалаврський) _____

Інтерактивний веб-додаток для навчання англійської мови із чат-ботом на базі AI.
Профіль користувача та реалізація тестування.

Виконав:
здобувач (ка) 3 курсу, групи ПЗП-22-9
_____ Вікторія СЕРДЮК _____

Спеціальність 121 – Інженерія програмного
забезпечення

Тип програми _____ освітньо-професійна _____
Освітня програма Програмна інженерія

Керівник ст.викл. кафедри ПІ Катерина ЗИБІНА

Члени комісії
доц. Олександр ВЕЧУР _____
доц. Наталя КРАВЕЦЬ _____
доц. Андрій РАБОТЯГОВ _____

2025 р.

Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук (або центр післядипломної освіти, або навчально-науковий центр заочної форми навчання) _____
 Кафедра _____ програмної інженерії _____
 Рівень вищої освіти _____ перший (бакалаврський) _____
 Спеціальність _____ 121 – Інженерія програмного забезпечення _____
 Тип програми _____ Освітньо-професійна _____
 Освітня програма _____ Програмна Інженерія _____

Курс 3Група ПЗП-22-9Семестр 6

ЗАВДАННЯ
на курсовий проект(роботу) студента

здобувачеві _____ Сердюк Вікторії Сергіївні _____

1. Тема роботи Інтерактивний веб-додаток для навчання англійської мови із чат-ботом на базі AI. Профіль користувача та реалізація тестування.

2. Термін здачі студентом закінченої роботи „ 19 ” червня 2025 р.

3. Вихідні дані до проєкту: електронні ресурси за обраною тематикою, мінімальні вимоги до функціональності програми, загальні вимоги до архітектури клієнтської частини системи, оптимізаційні методи та моделі, технології React.js, Node.js, Express, Sequelize

4. Перелік питань, що потрібно опрацювати в роботі

Аналіз предметної області, розробка постановки задачі, проєктування програмного забезпечення, програмна реалізація, аналіз результатів

Керівник роботи _____ ст.викл. кафедри ПІ Катерина ЗИБІНА

РЕФЕРАТ / ABSTRACT

Пояснювальна записка містить: 63 стор., 21 рис., 11 джерел.

АНГЛІЙСЬКА МОВА, ВЕБ-ДОДАТОК, ЧАТ-БОТ, CSS, EXPRESSJS, HTML, JAVASCRIPT, NODEJS, REACT, SQLITE

Об'єкт розробки – веб-додаток для навчання англійської мови з інтегрованим чат-ботом на основі штучного інтелекту.

Мета розробки – створення інтерактивного веб-застосунку, який забезпечить ефективне, персоналізоване вивчення англійської мови

Метод реалізації – розробка клієнт-серверної системи, де інтерфейс користувача реалізується за допомогою HTML, CSS, JavaScript та бібліотеки React[1]. Для збереження й обробки даних застосовується ORM Sequelize у зв'язці з Node.js та Express.js як серверною платформою. Для розширення функціональності використано Google Gemini – технологія, яка дозволяє реалізувати чат-бота, здатного відповідати на запитання користувача в реальному часі, генерувати фрази та допомагати в навчальному процесі.

У результаті розробки створено повнофункціональний веб-додаток, що дозволяє користувачу:

- проходити реєстрацію та авторизацію;
- взаємодіяти з персональним профілем (редагування, зміна пароля, видалення акаунту);
- вивчати лексику та додавати слова до словника;
- практикуватися у виконанні різноманітних вправ (тести, переклад, побудова речень);
- отримувати зворотній зв'язок через чат-бота з AI;

Застосунок в майбутньому може бути покращена за рахунок додавання панелі адміністратора, детальної аналітики результатів користувачів і інтеграції з MySQL.

ENGLISH LANGUAGE, WEB APPLICATION, CHATBOT, CSS, EXPRESSJS, HTML, JAVASCRIPT, NODEJS, REACT, SQLITE

The object of development is a web application for learning English with an integrated chatbot based on artificial intelligence.

The purpose of the work is creation of an interactive web application that will provide effective, personalized English language learning.

Solution method – development of a client-server system, where the user interface is implemented using HTML, CSS, JavaScript, and the React library. For data storage and processing, ORM Sequelize is used in conjunction with Node.js and Express.js as the server platform. To extend functionality, Google Gemini is used – a technology that allows the implementation of a chatbot capable of responding to user queries in real time, generating phrases, and assisting in the learning process.

As a result of the development, a full-featured web application has been created that allows the user to:

- complete registration and authorization;
- interact with a personal profile (edit data, change password, delete account);
- learn vocabulary and add words to a personal dictionary;
- practice through various types of exercises (tests, translation, sentence building);
- receive feedback through an AI-powered chatbot.

The application may be improved in the future by adding an admin panel, detailed analytics of user results, and integration with MySQL.

ЗМІСТ

Вступ.....	7
1 Аналіз предметної галузі	8
1.1 Аналіз предметної галузі.....	8
1.2 Виявлення та вирішення проблем	8
1.2.1 Цільова аудиторія.....	9
1.2.2 Монетизація	9
1.3 Аналіз аналогів програмного забезпечення.....	10
2 Постановка задачі.....	14
3 Архітектура та проєктування програмного забезпечення	16
3.1 UML проєктування ПЗ.....	16
3.2 Проєктування архітектури ПЗ	18
3.3 Проєктування структури зберігання даних.....	22
3.4 Приклади використаних алгоритмів та методів.....	23
3.5 Проєктування UI/UX.....	25
4 Опис прийнятих програмних рішень	30
5 Аналіз отриманих результатів.....	36
5.1 Порівняння результатів із початковими вимогами.....	36
5.2 Аналіз ефективності прийнятих рішень.....	36
5.3 Обмеження та недоліки.....	36
Висновки	38
Перелік джерел посилання	39
Додаток А Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ	40
Додаток Б Специфікація ПЗ	43
Додаток В Слайди презентації.....	49

ВСТУП

Темою комплексного курсового проекту є розробка інтерактивного веб-додатку для навчання англійської мови із чат-ботом на базі штучного інтелекту. Основним завданням розробки є створення програми, яка надає користувачеві змогу ефективно практикувати англійську мову через інтерактивні вправи, такі як: вивчення нових слів, вибір правильної відповіді, побудова речення, переклад слів, а також проходження міні-тестів відповідно до рівня володіння мовою.

Метою роботи є створення сучасного веб-застосунку з використанням технологій штучного інтелекту, що дозволяє персоналізувати процес навчання відповідно до потреб і рівня знань кожного користувача. Програма містить набір вправ, які можна обирати відповідно до свого рівня, а також особистий профіль із можливістю редагування персональних даних та перегляду словника.

Для реалізації інтерфейсу користувача використовувалися: HTML, CSS, JavaScript та бібліотека React. Бекенд побудовано на Node.js із застосуванням Express.js[2]. Для зберігання та обробки даних обрано ORM Sequelize у поєднанні з базою даних SQLite. Окрім того, використовується Google Gemini – AI-технологія, яка забезпечує функціонування чат-бота для підтримки навчального процесу.

Створення цього веб-додатку дозволить користувачам отримати доступ до зручного й ефективного інструменту вивчення англійської мови з будь-якого пристрою та у зручний для них час. Завдяки використанню технологій штучного інтелекту, користувачі зможуть отримувати миттєвий зворотний зв'язок, допомогу в навчанні та покращити свої навички за допомогою спілкування з чат-ботом.

У застосунку також передбачається розширення функціональних можливостей системи, а саме додавання нових типів інтерактивних вправ, удосконалення механізмів адаптивного навчання та впровадження детальної аналітики для відстеження прогресу користувачів. Запровадження таких функцій сприятиме підвищенню ефективності процесу вивчення англійської мови, забезпечуючи персоналізований підхід та мотивацію до навчання.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Аналіз предметної галузі

Сфера вивчення англійської мови стрімко розвивається завдяки впровадженню цифрових технологій. Попит на володіння англійською мовою постійно зростає в таких сферах, як бізнес, освіта, інформаційні технології, наука та багато інших.

Сучасні додатки для вивчення іноземних мов пропонують широкий спектр можливостей для покращення навичок читання, письма, аудіювання та словникового запасу. Але більшість наявних програм не надають користувачеві можливості повноцінної взаємодії з AI-помічником, який би адаптувався до рівня знань та цілей конкретного користувача.

Через це, було прийнято рішення розробити інтерактивний веб-додаток з використанням штучного інтелекту, який дозволяє ставити будь-які запитання, отримувати відповіді у режимі реального часу, а також вивчати лексику за темами відповідно до обраного рівня володіння мовою.

1.2 Виявлення та вирішення проблем

Завдяки розвитку технологій та можливості швидкого доступу до будь-яких програм, вивчення іноземних мов стало набагато простішим і доступнішим. Проте велика кількість додатків для вивчення англійської мови, мають деякі обмеження, що знижують ефективність навчання, а саме:

- обмежена адаптивність навчального контенту до рівня знань і інтересів користувача;
- відсутність або обмежене використання можливостей штучного інтелекту;
- неможливість отримати індивідуальні рекомендації в режимі реального часу.

Для вирішення зазначених недоліків необхідно створити веб-застосунок для вивчення англійської мови, який зосередиться на розв'язанні цих проблем.

Особливостями майбутнього застосунку будуть:

- адаптація навчальних матеріалів до рівня володіння мовою користувача;
- інтерактивні вправи різних типів;
- зручний інтерфейс із адаптивним дизайном;
- використання штучного інтелекту для надання зворотного зв'язку в режимі реального часу;

Таким чином, фінальний застосунок зможе усунути основні недоліки аналогів.

1.2.1 Цільова аудиторія

Основною цільовою аудиторією веб-додатку будуть:

- школярі;
- студенти;
- люди, що планують переїзд за кордон;
- люди, що планують подорожувати;
- фахівці у сфері IT, туризму, бізнесу;

Веб-додаток розрахований на користувачів з рівнем володіння мовою від початкового до просунутого, що дає можливість поступового розвитку навичок. Різні види вправ дають змогу адаптувати навчальний процес під індивідуальні потреби та особисті інтереси кожного користувача. Таким чином, платформа орієнтована як на тих, хто тільки починає вивчати англійську, так і на тих, хто прагне удосконалити вже наявні знання.

1.2.2 Монетизація

Уся функціональність веб-додатку для вивчення англійської мови з використанням штучного інтелекту наразі є безкоштовним. Але, у майбутньому планується поступове впровадження монетизації:

- збереження основних функцій у безкоштовному доступі;
- персоналізовані рекомендації та статистики прогресу за передплатою;
- розширене використання функцій штучного інтелекту.

Монетизація планується таким чином, щоб не обмежувати основний функціонал для користувачів, а розвивати проєкт та підвищувати його якість.

1.3 Аналіз аналогів програмного забезпечення

Для аналізу аналогів був обраний «Busuu» (див. рис. 1.1). Після вибору бажаної мови та встановлення рівня володіння, користувач отримує доступ до структурованих уроків, які поступово розвивають різні мовні навички[3].

Основні функціональні можливості додатку:

- індивідуальний навчальний план, що адаптується під обраний рівень;
- інтерактивні вправи на читання, граматику, письмо та аудіювання;
- відео та діалоги для практики розмовної мови;
- можливість перевірити свої знання за допомогою вправ та тестів;
- особистий кабінет з прогресом та статистикою;
- опція перевірки письмових завдань іншими користувачами.

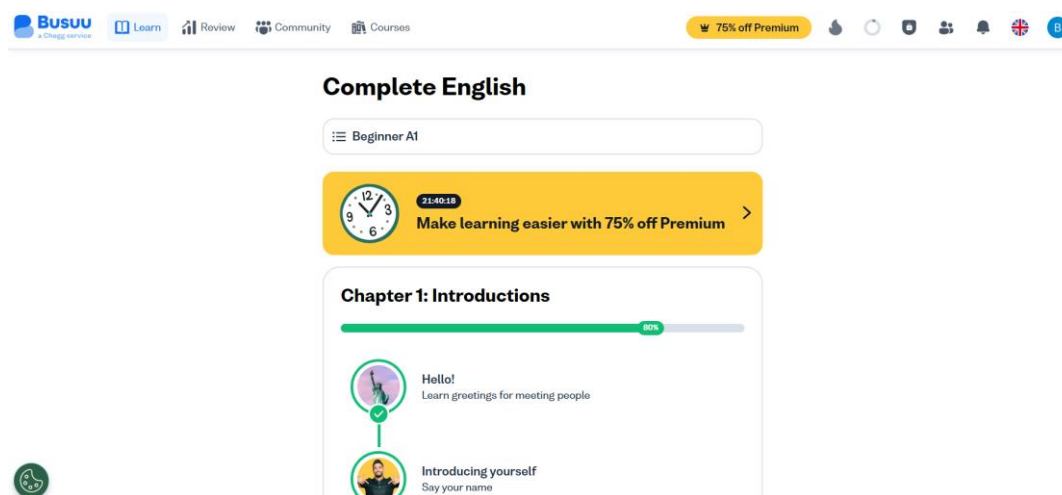


Рисунок 1.1 – Веб-додаток Busuu

Перевагами програми «Busuu» є :

- структуровані уроки відповідно до рівня користувача;
- присутні короткі відео для покращення навичок слухання;

- детальне пояснення граматичних правил та правопису після кожного завдання;
- можливість перегляду пройденої граматики та вивчених слів;
- можливість проходження різних навчальних курсів за тематичними напрямками (див. рис. 1.2).

Серед недоліків можна виділити - обмежений доступ до курсів середнього та просунутого рівня без оформлення платної підписки.

More courses

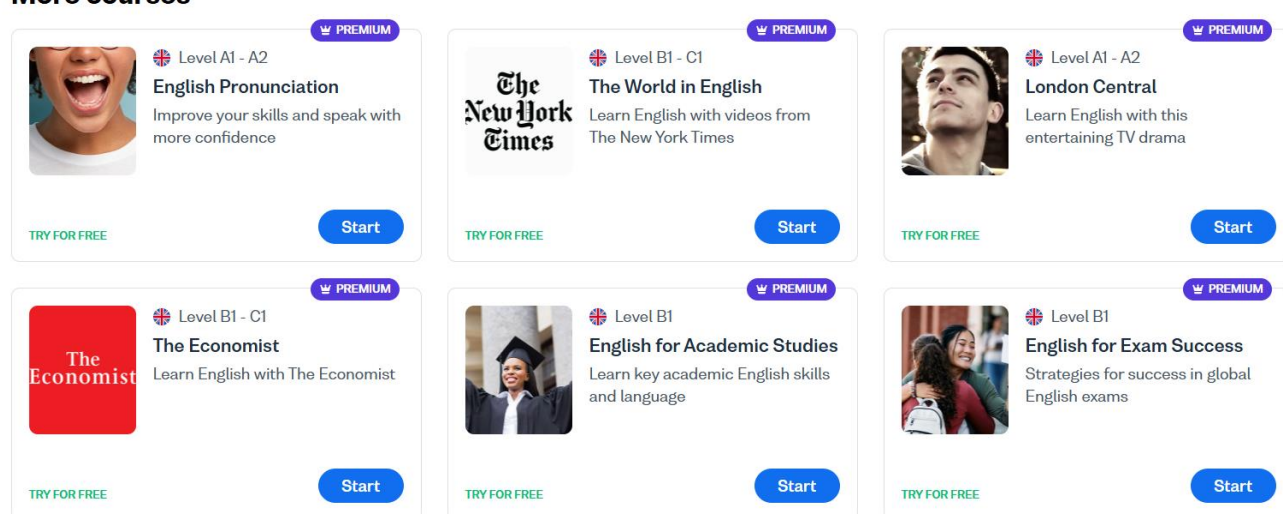


Рисунок 1.2 – Курси додатку Busuu

Інший застосунок для вивчення мов: «Memrise» (див. рис. 1.3). Програма спеціалізується на запам'ятовуванні лексики через інтерактивні картки та відео з носіями мови і пропонує великий вибір курсів, орієнтованих на різні теми та рівні володіння мовою[6].

Функціональні можливості додатку:

- візуальні картки зі словами, перекладом та вимовою;
- відео за участю носіїв мови, де вони використовують слова в реальних ситуаціях;

- повторення вивченого матеріалу за принципами інтервального повторення;
- особистий словник і щоденні цілі;
- ігрові елементи для мотивації: рівні, нагороди;
- інтерактивна взаємодія з чат-ботом, який пропонує індивідуальні навчальні завдання відповідно до рівня та тематики, що цікавить користувача (див. рис. 1.4).

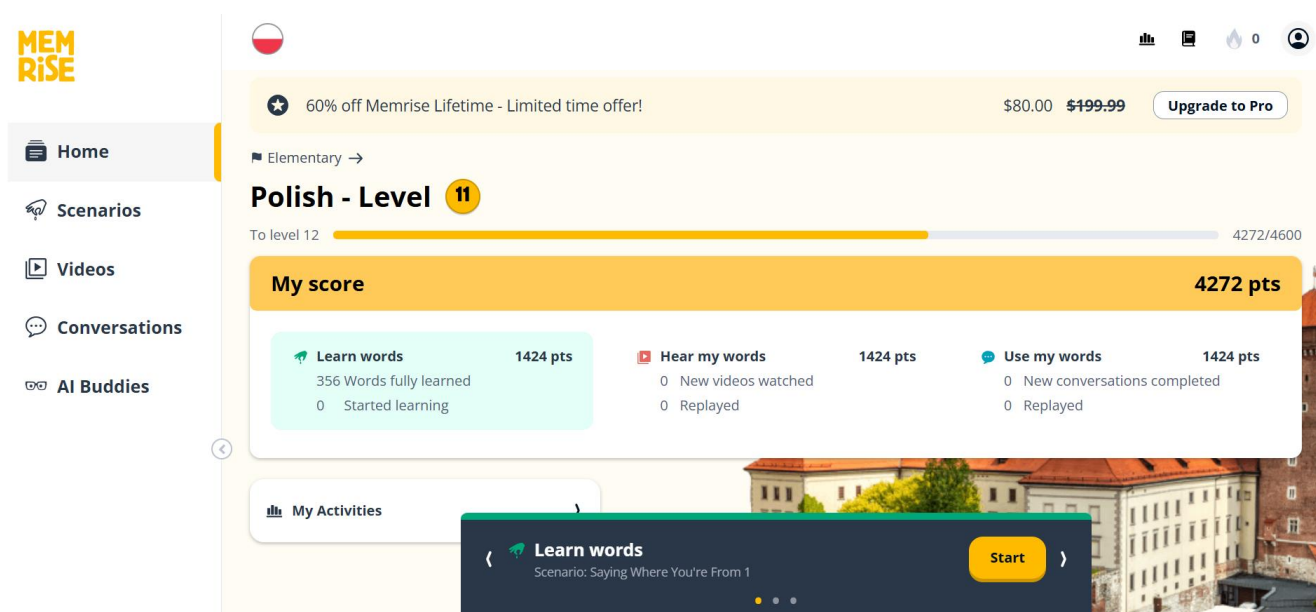


Рисунок 1.3 – Веб-додаток Memrise

Перевагами програми «Memrise» є :

- запам'ятовування слів та фраз за допомогою інтерактивних карток;
- великий вибір курсів, орієнтованих на різні теми та рівні володіння мовою;
- відео з носіями мови для кращого занурення в мовне середовище;

В якості недоліку можна виділити - відсутність детальних пояснень правил граматики.

Your Buddies Beta

Supporting you with your learning journey

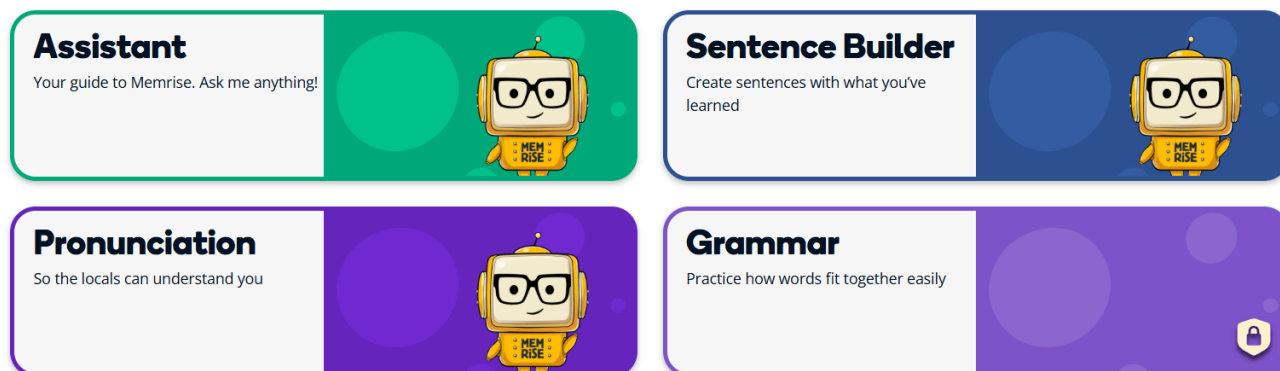


Рисунок 1.4 – Чат-боти додатку Memrise

У межах даного розділу було проаналізовано програмні рішення для вивчення іноземних мов: Busuu та Memrise, які пропонують широкий набір можливостей для розвитку мовних навичок.

Обидва додатки пропонують адаптивне навчання, використання мультимедійних матеріалів та елементи гейміфікації, що сприяє підвищенню мотивації користувачів. Проведений аналіз дав змогу виявити й певні недоліки, зокрема обмежений безкоштовний доступ до частини навчального контенту та відсутність детальних граматичних пояснень.

З урахуванням переваг і недоліків розглянутих аналогів, наша розробка буде спрямована на усунення виявлених проблем і поєднання сильних сторін в одному інтерактивному рішенні. Особливу увагу планується приділити збереженню доступності основного функціоналу та гнучкій адаптації до рівня користувача. Очікується, що створювана система стане ефективним, зручним і сучасним інструментом для вивчення англійської мови.

2 ПОСТАНОВКА ЗАДАЧІ

У межах комплексного курсового проєкту передбачено розробку програмної системи “Інтерактивний веб-додаток для навчання англійської мови із чат-ботом на базі AI. Профіль користувача та реалізація тестування”, яка має на меті створити зручний і адаптивний інструмент для ефективного вивчення англійської мови. Dodatok dozwolity korystuwacham praktykuvaty movu cherez riznomanitni vidi vprav ta otrymuvaty personalizovanu pidtrimku u navchanni.

Система має бути реалізована як клієнт-серверний додаток з веб-інтерфейсом, доступним у різних браузерях, таких як Google Chrome та Microsoft Edge, з адаптивним дизайном для коректного відображення та функціонування на різних пристроях. Для зберігання даних користувачів, результатів тестів, словників та іншої інформації необхідно розробити базу даних, яка забезпечить безпечне та ефективне збереження і обробку інформації.

Необхідно реалізувати такі функціональні модулі застосунку:

а) модуль головної сторінки:

- 1) призначений для першого знайомства користувача з платформою;
- 2) має містити короткий опис мети додатку та функціональностей;
- 3) має містити зручну навігацію до інших сторінок.

б) модуль реєстрації:

- 1) для реєстрації у застосунку користувач вводить ім'я, прізвище, електронну пошту, номер телефону, пароль та рівень володіння англійською мовою;
- 2) присутній вивід помилок про невірно введені дані, занадто простий пароль або ж, що така пошта вже зареєстрована;
- 3) пароль користувача у базі даних повинен проходити процес хешування.

в) модуль авторизації:

- 1) форма для вводу полів електронної пошти та пароля;
- 2) присутній вивід помилок про невірно введені дані.

г) модуль профілю користувача:

- 1) є можливість зберігання та редагування таких даних, як: ім'я, прізвище, номер телефону, електронна пошта, рівень володіння мовою;
- 2) функція зміни пароля: для цього користувач повинен ввести свій поточний пароль, а також двічі новий пароль, щоб уникнути помилок при введенні;
- 3) функція виходу з облікового запису;
- 4) функція видалення облікового запису;
- 5) зручна навігація до інших функцій програми.

д) модуль міні-тестів:

- 1) користувач повинен мати можливість обрати тест відповідно до свого рівня володіння англійською (початковий, середній або просунутий);
- 2) обмеження часу на проходження - 7 хвилин;
- 3) кожен тест включатиме 10 питань;
- 4) після завершення тесту відбуватиметься автоматична перевірка відповідей із підрахунком правильних результатів;
- 5) є можливість пройти тест повторно для покращення результату.

е) модуль словника:

- 1) користувач повинен мати можливість переглянути додані слова для вивчення;
- 2) є можливість написати переклад англійського слова;
- 3) є можливість перевірити правильність перекладу слова.

3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 UML проєктування ПЗ

Для веб-додатку з вивчення англійської мови була створена діаграма use-case, яка відображає основні сценарії взаємодії користувача з системою. (див. рис. 3.1).

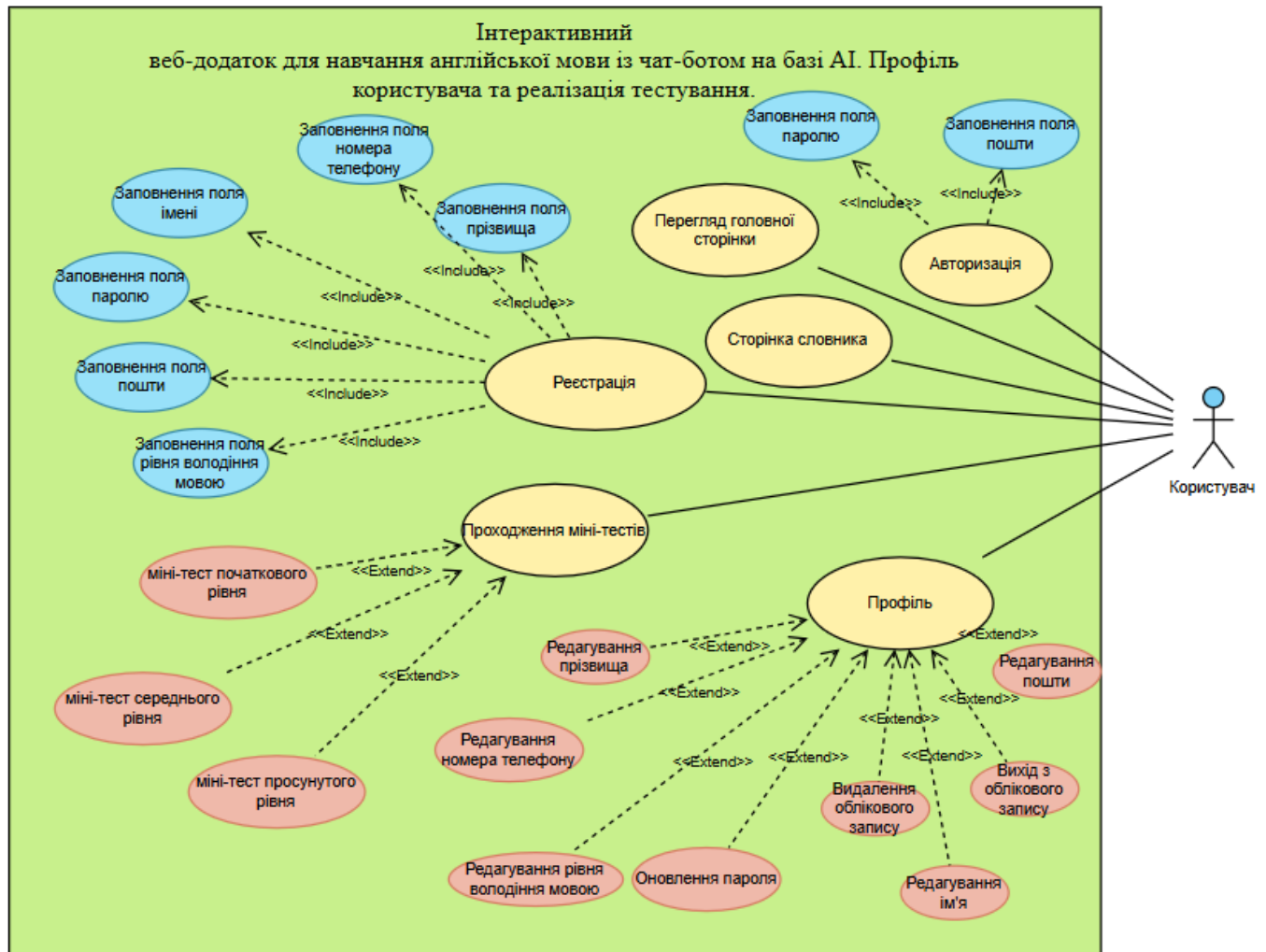


Рисунок 3.1 – Use-case діаграма інтерактивного веб-додатку для навчання англійської мови (рисунок виконаний самостійно)

На діаграмі зображені основні функціональні можливості додатку. Єдиним актором у системі є користувач, який має можливість авторизуватися та зареєструватися в системі.

Під час реєстрації обов'язковим є заповнення таких полів, як: ім'я, прізвище, електронна пошта, номер телефону, пароль та рівень володіння англійською мовою. Для авторизації достатньо ввести адресу електронної пошти та пароль.

Після успішної реєстрації або входу до системи користувач отримує доступ до головної сторінки, на якій представлено загальну інформацію про платформу, її функціональність та переваги.

Додаток надає можливість проходити міні-тести, які розділені за рівнями складності: початковий, середній та просунутий. Кожен тест містить 10 питань із таймером, після завершення яких користувач одразу отримує результат.

Користувач також може перейти до особистого профілю, де передбачена можливість редагування персональних даних, таких як: ім'я, прізвище, номер телефона, рівень англійської мови, електронна пошта та пароль. У профілі користувача також є можливість вийти з облікового запису або повністю його видалити.

Із профілю реалізовано перехід до словника, в якому користувач може переглядати слова, додані для вивчення, вводити їхній переклад та перевіряти правильність власної відповіді.

Також була створена діаграма діяльності (див. рис. 3.2).

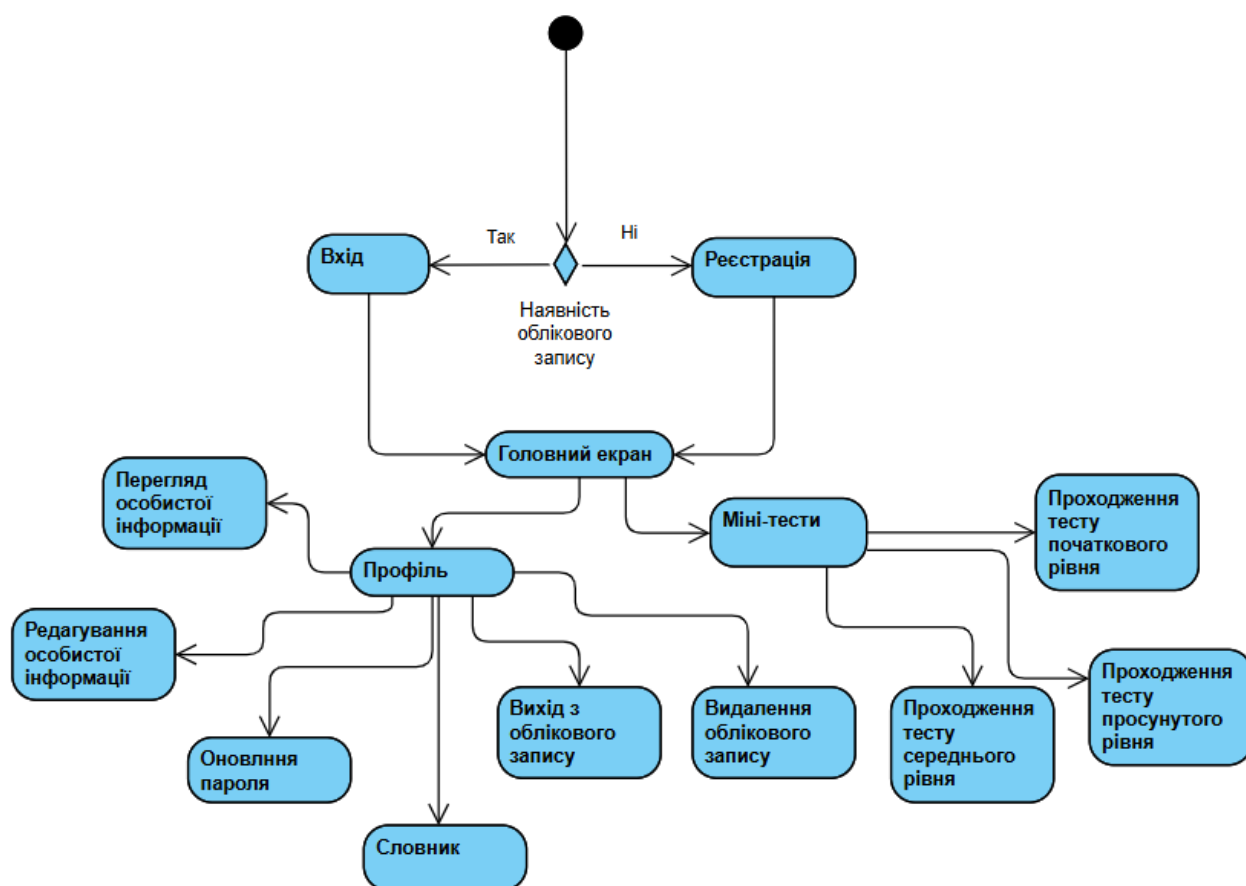


Рисунок 3.2 – Діаграма діяльності інтерактивного веб-додатку для навчання англійської мови (рисунок виконаний самостійно)

Діаграма діяльності ілюструє послідовність основних кроків, які виконує користувач у системі: від реєстрації та авторизації до взаємодії з головною сторінкою, проходження міні-тестів, редагування профілю та роботи зі словником. Вона чітко відображає логіку переходів між цими етапами і демонструє основні варіанти дій користувача.

3.2 Проектування архітектури ПЗ

Метою створення веб-додатку є забезпечення ефективного інструменту для вивчення англійської мови.

Призначення системи: надати користувачам можливість проходити міні-тести, додавати нові слова до словника та керувати власним профілем.

Зацікавлені сторони: користувачі, а саме: учні, студенти, мандрівники, фахівці, розробники, адміністратор системи.

Архітектурний огляд:

Для проєкту було обрано архітектурний стиль клієнт-сервер. Загальна архітектура містить три основні шари:

- рівень презентації: відповідає за інтерфейс користувача, реалізовано за допомогою React[9];
- рівень логіки застосунку: реалізований на Node.js з використанням Express.js;
- рівень доступу до даних: використовується локальна база даних SQLite.

Компоненти системи:

Frontend:

- реєстрація;
- авторизація;
- головна сторінка;
- міні-тести;
- профіль;
- словник.

Backend:

- авторизація та аутентифікація;
- обробка тестів;
- збереження та редагування даних профілю.

База даних:

- зберігання інформації про користувачів та міні-тести.

Комунікація між компонентами:

Взаємодія між клієнтом і сервером здійснюється через REST API. Для передачі даних використовується HTTP/HTTPS, а структура запитів - JSON.

Була створена діаграма компонентів (див. рис. 3.3).

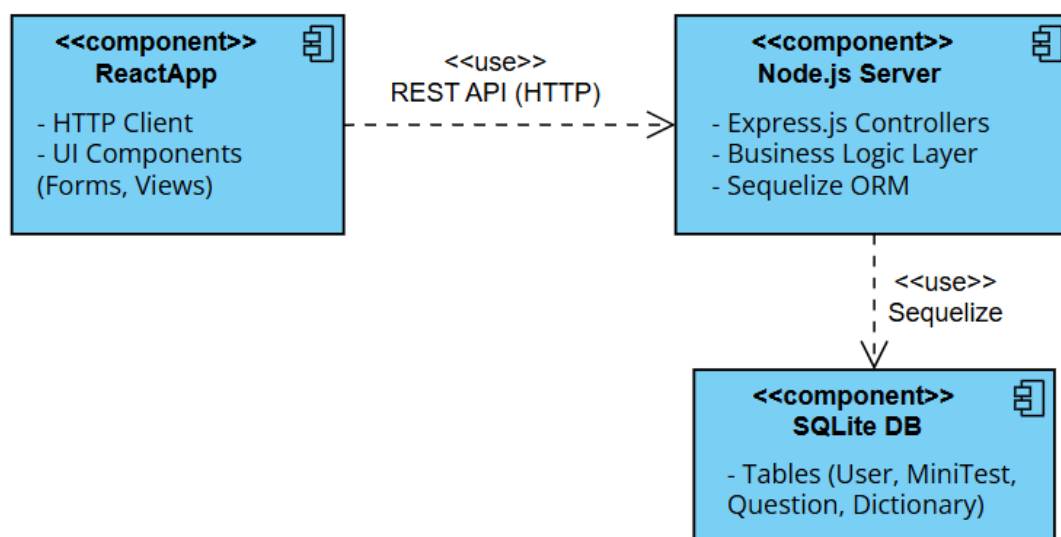


Рисунок 3.3 – Діаграма компонентів інтерактивного веб-додатку для навчання англійської мови (рисунок виконаний самостійно)

Діаграма компонентів відображає структуру програмної системи та взаємодію її складових частин. Першим компонентом є React-App, який відповідає за інтерфейс користувача. Він містить два основні підкомпоненти: HTTP Client, що забезпечує зв'язок із сервером, та UI Components, які реалізують форми та візуальні елементи для взаємодії користувача з додатком. За допомогою REST API через HTTP-запити, React-App взаємодіє з компонентом Node.js Server, який реалізує бекенд-логіку[7].

У компоненті Node.js Server знаходяться Express.js Controllers, що відповідають за обробку вхідних запитів і маршрутизацію, Business Logic Layer, де реалізовані основні алгоритми та логіка обробки даних, а також Sequelize ORM, який служить посередником для роботи з базою даних.

Для збереження інформації використовується компонент SQLite DB, що містить кілька таблиць: User, MiniTest, Question та Dictionary. Сервер через ORM здійснює читання, запис і оновлення даних у цих таблицях, забезпечуючи безпечне та ефективне управління інформацією.

Також була створена діаграма розгортання (див. рис. 3.4).

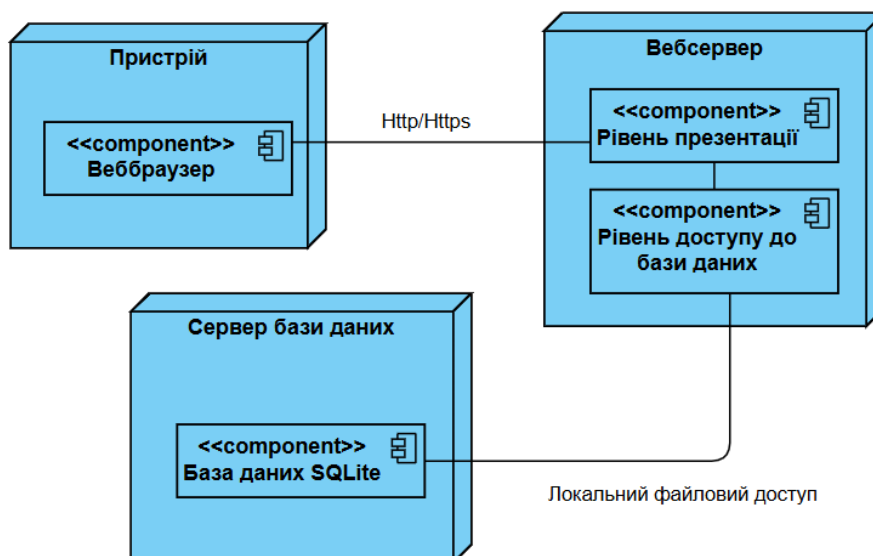


Рисунок 3.4 – Діаграма розгортання інтерактивного веб-додатку для навчання англійської мови (рисунок виконаний самостійно)

Діаграма розгортання демонструє фізичне розміщення компонентів системи та їх взаємодію. Користувач отримує доступ до системи через веб-браузер, запущений на клієнтському пристрої. Через нього надсилаються HTTP/HTTPS запити до веб-сервера, на якому реалізована логіка застосунку.

На стороні веб-сервера присутні такі основні компоненти:

- рівень презентації, який відповідає за обробку запитів від користувача та відправку відповідей;
- рівень доступу до бази даних, що реалізує бізнес-логіку додатку та забезпечує з'єднання з базою даних.

База даних реалізована з використанням SQLite, а доступ до неї здійснюється через локальний файловий інтерфейс.

Нефункціональні вимоги:

Продуктивність: швидкий відгук на дії користувача, до 2 секунд.

Масштабованість: заміна SQLite на MySQL;

Надійність: регулярне резервне копіювання;

Безпека: хешування паролів та валідація введених даних.

Оцінка ризиків: Нестабільна робота SQLite при великому навантаженні може бути усунена завдяки переходу на MySQL.

Таким чином, була розроблена архітектура програмного забезпечення, що базується на клієнт-серверній взаємодії та забезпечує зручність використання. Реалізовані діаграми ілюструють структуру, поведінку та взаємозв'язки основних елементів системи, які слугують основою для реалізації функціональних та нефункціональних вимог застосунку.

3.3 Проектування структури зберігання даних

База даних веб-додатку для вивчення англійської мови створена для зберігання інформації про користувачів, тестів та словника. Побудована на основі реляційної моделі з використанням SQLite.

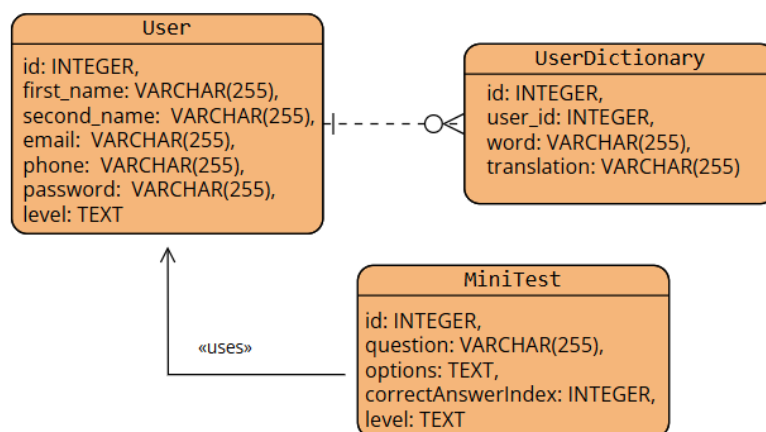


Рисунок 3.5 – Схема бази даних (рисунок виконаний самостійно)

На схемі зображено основні таблиці бази даних: User з даними про користувачів, UserDictionary зі словами, що були додані користувачами для вивчення, і MiniTest з питаннями для тестів. Між User і UserDictionary встановлено зв'язок “один-до-багатьох”, бо один користувач може мати декілька записів у словнику. Зв'язок “uses” між MiniTest і User вказує на те, що користувачі проходять тести, хоча фізичного зв'язку в базі даних немає.

3.4 Приклади використаних алгоритмів та методів

Функція валідації форми реєстрації:

```
const validateForm = () => {
  const nameRegex = /^[a-zA-Za-яA-ЯіієіїЄІ'-\s]+$/;

  if (!formData.first_name.trim()) {
    return "Введіть ім'я";
  }

  if (!nameRegex.test(formData.first_name)) {
    return "Ім'я може містити лише літери";
  }

  if (!formData.second_name.trim()) {
    return "Введіть прізвище";
  }

  if (!nameRegex.test(formData.second_name)) {
    return "Прізвище може містити лише літери";
  }

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(formData.email)) {
    return "Введіть коректний email";
  }

  const phoneRegex = /^[+()\-0-9\s]{6,15}$/;
  if (!phoneRegex.test(formData.phone)) {
    return "Введіть коректний номер телефону";
  }

  if (formData.password.length < 6) {
    return "Пароль повинен містити мінімум 6 символів";
  }

  if (!formData.level) {
    return "Будь ласка, оберіть рівень";
  }
}
```

```

    return null;
};

```

Функція `validateForm` здійснює перевірку правильності введених даних у форму реєстрації. Вона перевіряє, чи заповнені обов'язкові поля, чи ім'я та прізвище містять тільки літери, довжину пароля і чи вибрано рівень знання мови[8].

```

const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setSuccess("");

    const validationError = validateForm();
    if (validationError) {
        setError(validationError);
        return;
    }

    try {
        const response = await
fetch("http://localhost:3001/api/auth/register", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(formData),
        });

        const data = await response.json();

        if (response.ok) {
            setSuccess("Реєстрація успішна!");
            setFormData({
                first_name: "",
                second_name: "",
                email: "",
                phone: "",
            });
        }
    } catch (error) {
        setError(error.message);
    }
};

```



```

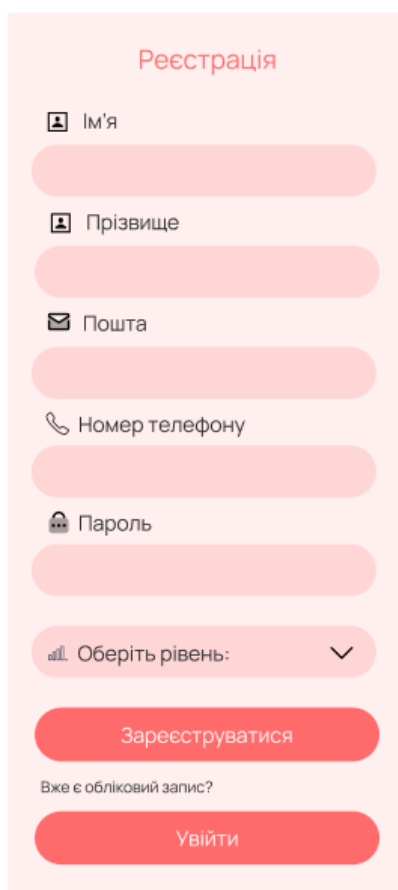
        password: "",
        level: "",
    });
    setTimeout(() => navigate("/login"), 2000);
} else {
    setError(data.error || "Помилка реєстрації");
}
} catch (err) {
    setError("Помилка сервера");
}
};

```

Функція `handleSubmit` обробляє відправку форми: спочатку перевіряє дані, потім, відправляє POST-запит до серверного API, з даними реєстрації у форматі JSON. Після отримання відповіді від сервера користувачу відображається повідомлення про успішну реєстрацію або помилку, а в разі успіху, здійснюється переадресація на сторінку входу.

3.5 Проєктування UI/UX або іншого дизайну системи

Було розроблено макети сторінок (реєстрації, входу, головної сторінки, словнику, оновлення пароля, профілю користувача, загальної сторінки міні-тестів та проходження міні-тесту початкового рівня) веб-додатку у середовищі Figma (див. рис. 3.6 – 3.13).



Реєстрація

Ім'я

Прізвище

Пошта

Номер телефону

Пароль

Оберіть рівень: ▼

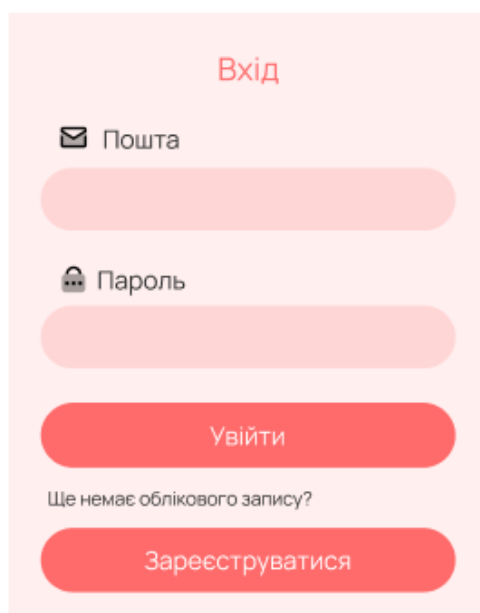
Зареєструватися

Вже є обліковий запис?

Увійти

The registration form is titled 'Реєстрація' in red. It contains six input fields with corresponding icons: a person icon for 'Ім'я', another person icon for 'Прізвище', an envelope icon for 'Пошта', a telephone handset icon for 'Номер телефону', a padlock icon for 'Пароль', and a signal strength icon for 'Оберіть рівень:'. Below the fields is a red 'Зареєструватися' button. At the bottom, there is a link 'Вже є обліковий запис?' and a red 'Увійти' button.

Рисунок 3.6 – Дизайн форми реєстрації (рисунок виконаний самостійно)



Вхід

Пошта

Пароль

Увійти

Ще немає облікового запису?

Зареєструватися

The login form is titled 'Вхід' in red. It contains two input fields with corresponding icons: an envelope icon for 'Пошта' and a padlock icon for 'Пароль'. Below the fields is a red 'Увійти' button. At the bottom, there is a link 'Ще немає облікового запису?' and a red 'Зареєструватися' button.

Рисунок 3.7 – Дизайн форми входу (рисунок виконаний самостійно)



Рисунок 3.8 – Дизайн головної сторінки застосунку (рисунок виконаний самостійно)

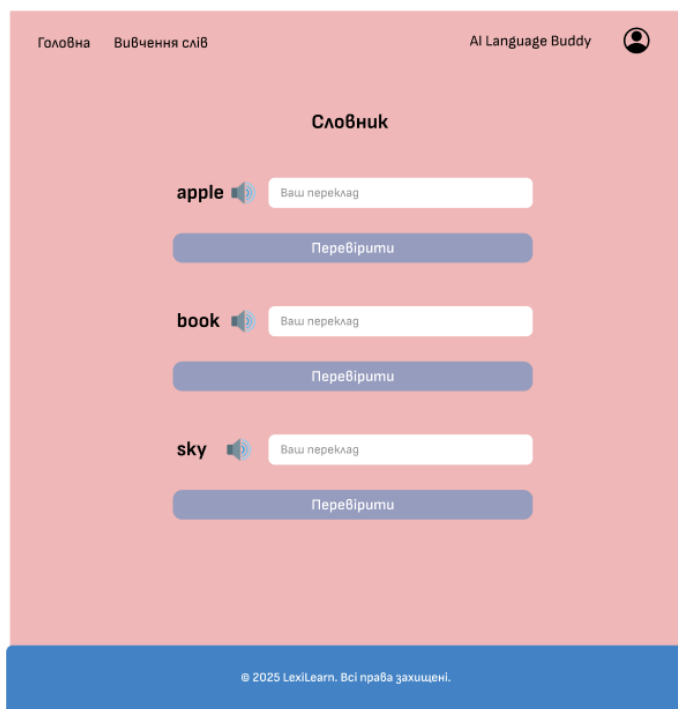


Рисунок 3.9 – Дизайн словника застосунку (рисунок виконаний самостійно)



Головна Вивчення слів Словник

Профіль користувача

Ім'я: Прізвище:

Email:

Телефон: Рівень:

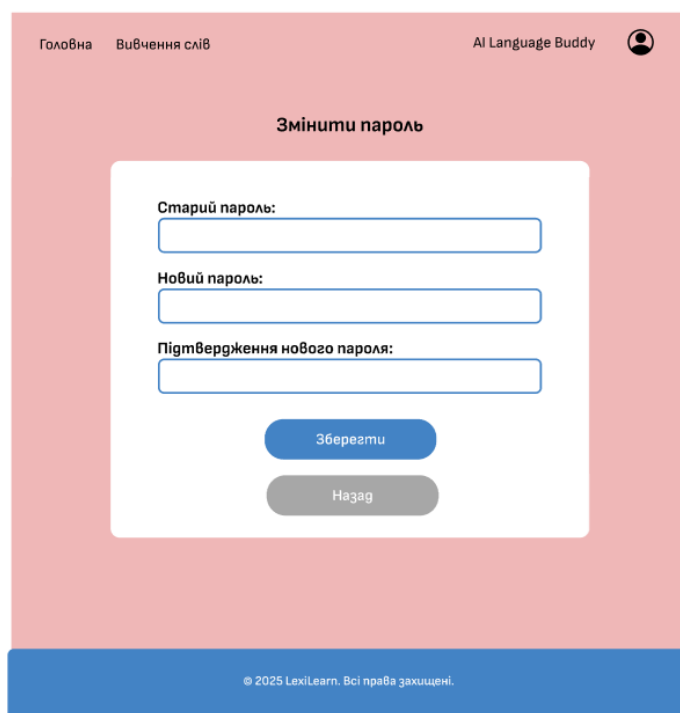
[Змінити пароль](#)

[Зберегти зміни](#)

[Вийти з акаунту](#) [Видалити акаунт](#)

© 2025 LexiLearn. Всі права захищені.

Рисунок 3.10 – Дизайн профілю користувача (рисунок виконаний самостійно)



Головна Вивчення слів AI Language Buddy 

Змінити пароль

Старий пароль:

Новий пароль:

Підтвердження нового пароля:

[Зберегти](#)

[Назад](#)

© 2025 LexiLearn. Всі права захищені.

Рисунок 3.11 – Дизайн функції зміни пароля у застосунку (рисунок виконаний самостійно)



Рисунок 3.12 – Дизайн сторінки міні-тесту (рисунок виконаний самостійно)



Рисунок 3.13 – Дизайн проходження міні-тесту початкового рівня (рисунок виконаний самостійно)

У процесі проєктування UI/UX було приділено увагу створенню інтуїтивно зрозумілого, зручного та адаптивного інтерфейсу, для забезпечення комфортної взаємодії користувача із застосунком на різних пристроях. Макети сторінок розроблено з урахуванням сучасних тенденцій дизайну та принципів юзабіліті, що сприяють ефективному навчанню та мотивують користувачів до регулярного використання платформи.

4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ

Форма реєстрації дозволяє новому користувачу створити обліковий запис, ввівши: ім'я, прізвище, email, номер телефону, пароль і рівень володіння англійською мовою. Присутнє хешування паролю за допомогою бібліотеки bcrypt та перевірки правильності введених даних[5]. У разі, якщо користувач із вказаною електронною поштою вже існує в базі даних, система повертає повідомлення про помилку. Після успішної реєстрації дані користувача зберігаються у базі даних SQLite (див. рис. 4.1).

```

    router.post("/register", async (req, res) => {
      console.log("Received /register request with body:", req.body);

      const { first_name, second_name, email, phone, password, level } =
        req.body;

      if (!first_name || !second_name || !email || !phone || !password ||
        !level) {
        return res
          .status(400)
          .json({ error: "Всі поля є обов'язковими для заповнення" });
      }

      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      if (!emailRegex.test(email)) {
        return res.status(400).json({ error: "Неправильний формат email" });
      }

      const phoneRegex = /^[0-9+\-\s()]*$/;
      if (!phoneRegex.test(phone)) {
        return res.status(400).json({ error: "Неправильний формат телефону" });
      }

      const allowedLevels = ["початковий", "середній", "просунутий"];
      if (!allowedLevels.includes(level)) {
        return res
          .status(400)
          .json({ error: "Неправильне значення рівня (level)" });
      }

      try {
        const existingUser = await User.findOne({ where: { email } });
        if (existingUser) {
          return res
            .status(400)
            .json({ error: "Користувач з таким email вже існує" });
        }

        const hashedPassword = await bcrypt.hash(password, 10);

```

```

const user = await User.create({
  first_name,
  second_name,
  email,
  phone,
  password: hashedPassword,
  level,
});

console.log("User registered successfully:", user.id);
res.status(201).json({ id: user.id });
} catch (err) {
  console.error("Register error details:", err);
  res.status(500).json({
    error: "Помилка сервера при реєстрації користувача",
    details: err.message,
  });
}
});

```

Рисунок 4.1 – Реєстрація користувача (рисунок виконаний самостійно)

За допомогою авторизації, вже зареєстрований користувач має можливість увійти до системи використовуючи пошту та пароль[10]. Після введення даних відбувається перевірка існування користувача в базі даних, а також порівняння збереженого пароля в БД із введеним паролем. У разі успішної авторизації генерується токен, який містить ідентифікатор користувача. У випадку невірно

введених даних, можна побачити відповідне повідомлення про помилку (див. рис. 4.2).

Код для сторінки авторизації:

```
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ where: { email } });

    if (!user) {
      return res.status(401).json({ error: "Невірна пошта або пароль" });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.status(401).json({ error: "Невірна пошта або пароль" });
    }

    const token = jwt.sign({ id: user.id }, process.env.SECRET_KEY, {
      expiresIn: "1h",
    });

    res.status(200).json({
      message: "Успішний вхід",
      token,
      user: {
        id: user.id,
        first_name: user.first_name,
        second_name: user.second_name,
        email: user.email,
      },
    });
  } catch (err) {
    console.error("Login error:", err);
    res.status(500).json({ error: "Помилка сервера під час входу" });
  }
});
```


Рисунок 4.2 – Авторизація користувача (рисунок виконаний самостійно)

Опис: у користувача також є можливість оновити пароль, для цього необхідно ввести старий пароль і новий. Сервер перевіряє, чи введено обидва значення, чи користувач існує, та чи співпадає старий пароль з хешованим, який зберігається в базі даних. Якщо перевірки проходять успішно, новий пароль хешується за допомогою бібліотеки `bcrypt` і зберігається в базі даних (див. рис. 4.3).

Код для сторінки оновлення пароля:

```
router.post("/change-password", authenticate, async (req, res) => {
  try {
    const userId = req.user.id;
    const { oldPassword, newPassword } = req.body;

    if (!oldPassword || !newPassword) {
      return res.status(400).json({ message: "Вкажіть обидва паролі" });
    }

    const user = await User.findByPk(userId);
    if (!user) {
      return res.status(404).json({ message: "Користувача не знайдено" });
    }

    const isMatch = await bcrypt.compare(oldPassword, user.password);
```

```

if (!isMatch) {
  return res.status(400).json({ message: "Старий пароль невірний" });
}

const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(newPassword, salt);
user.password = hashedPassword;
await user.save();

res.json({ message: "Пароль успішно змінено" });
} catch (err) {
  console.error(err);
  res.status(500).json({ message: "Помилка сервера" });
}
});

```

Рисунок 4.3 – Оновлення пароля користувача (рисунок виконаний самостійно)

Опис: За допомогою маршруту `/beginner` сервер виконує запит до бази даних та отримує 10 випадкових питань початкового рівня (див. рис. 4.4).

Код для сторінки міні-тесту початкового рівня:

```

router.get("/beginner", async (req, res) => {
  const level = req.query.level || "початковий";
  try {
    const questions = await MiniTest.findAll({
      where: { level },
      attributes: ["question", "options", "correctAnswerIndex"],

```

```

        order: Sequelize.literal("RANDOM()"),
        limit: 10,
    });
    res.json(questions);
} catch (error) {
    res.status(500).json({ error: "Помилка сервера" });
}
});

```

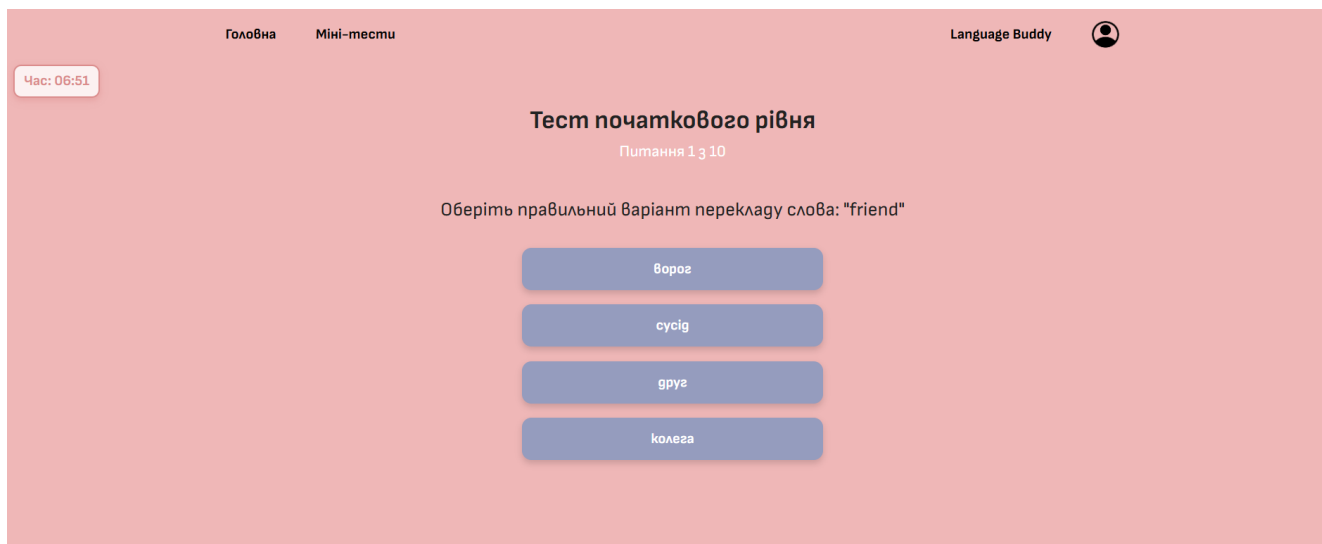


Рисунок 4.4 – Міні-тести для початкового рівня (рисунок виконаний самостійно)

Таким чином, у цьому розділі розглянуто реалізацію ключових функцій системи: реєстрації, авторизації, оновлення пароля та взаємодії з міні-тестами. Наведені приклади коду демонструють логіку обробки запитів, валідацію даних і взаємодію з базою даних.

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

5.1 Порівняння результатів із початковими вимогами

Під час реалізації застосунка були створені такі компоненти:

- реєстрація та авторизація із валідацією даних;
- головна сторінка із детальною інформацією застосунку;
- профіль користувача, що надає можливість редагування особистої інформації, виходу та видалення облікового запису, а також оновлення пароля;
- словник, для збереження нових слів, які користувач планує вивчати у майбутньому;
- міні-тести, для перевірки знань та закріплення вивченого матеріалу.

Таким чином, можна зробити висновок, що застосунок відповідає всім початковим вимогам.

5.2 Аналіз ефективності прийнятих рішень

Для забезпечення безпеки особистих даних користувачів була використана бібліотека `bcrypt`, яка дозволила хешувати паролі перед збереженням у базі даних. Ще одним важливим рішенням стало використання бібліотеки `jsonwebtoken`, яка дозволяє генерувати токени після авторизації. Завдяки цьому користувачі можуть залишатися авторизованими протягом години без необхідності повторного введення даних. Також, для взаємодії із базою даних було обрано `Sequelize ORM`, що дозволило значно спростити процес створення, налаштування та взаємозв'язку моделей.

5.3 Обмеження та недоліки

Незважаючи на реалізовані функції, у програмі присутні такі недоліки:

- відсутність панелі адміністратора, яка б дозволила додавати, редагувати та видаляти питань до тестів через застосунок;
- неможливість управління користувачами, а саме перегляду їх активності;

- відсутність збереження історії проходження тестів, що ускладнює відстеження прогресу користувача та аналіз ефективності навчання;
- відсутність статистики використання платформи у профілі користувача;

Незважаючи на ці обмеження, весь основний функціонал програми працює коректно.

ВИСНОВКИ

Застосунок інтерактивний веб-додаток для навчання англійської мови із чат-ботом на базі AI був успішно реалізований. В результаті, були створені такі сторінки:

- головна сторінка із коротким описом додатку, його функціоналу та навігацією до інших сторінок;
- сторінка реєстрації, де користувачу необхідно ввести ім'я, прізвище, електронну пошту, номер телефону, пароль та рівень володіння англійською мовою. Також присутній вивід помилок про невірно введені дані;
- сторінка авторизації, де користувачу необхідно ввести електронну пошту та пароль;
- профіль користувача, у якому можна редагувати особисті дані оновити пароль, вийти або видалити обліковий запис;
- сторінка міні-тестів, для перевірки вивченого матеріалу із обмеженням в часі – 7 хвилин, та питань – 10;
- сторінка словнику, для повторення доданих слів на вивчення.

Для реалізації були використані такі технології для інтерфейсу користувача: React, для роботи з базою даних: Sequelize та для бекенду: Node.js[4] та Express.js.

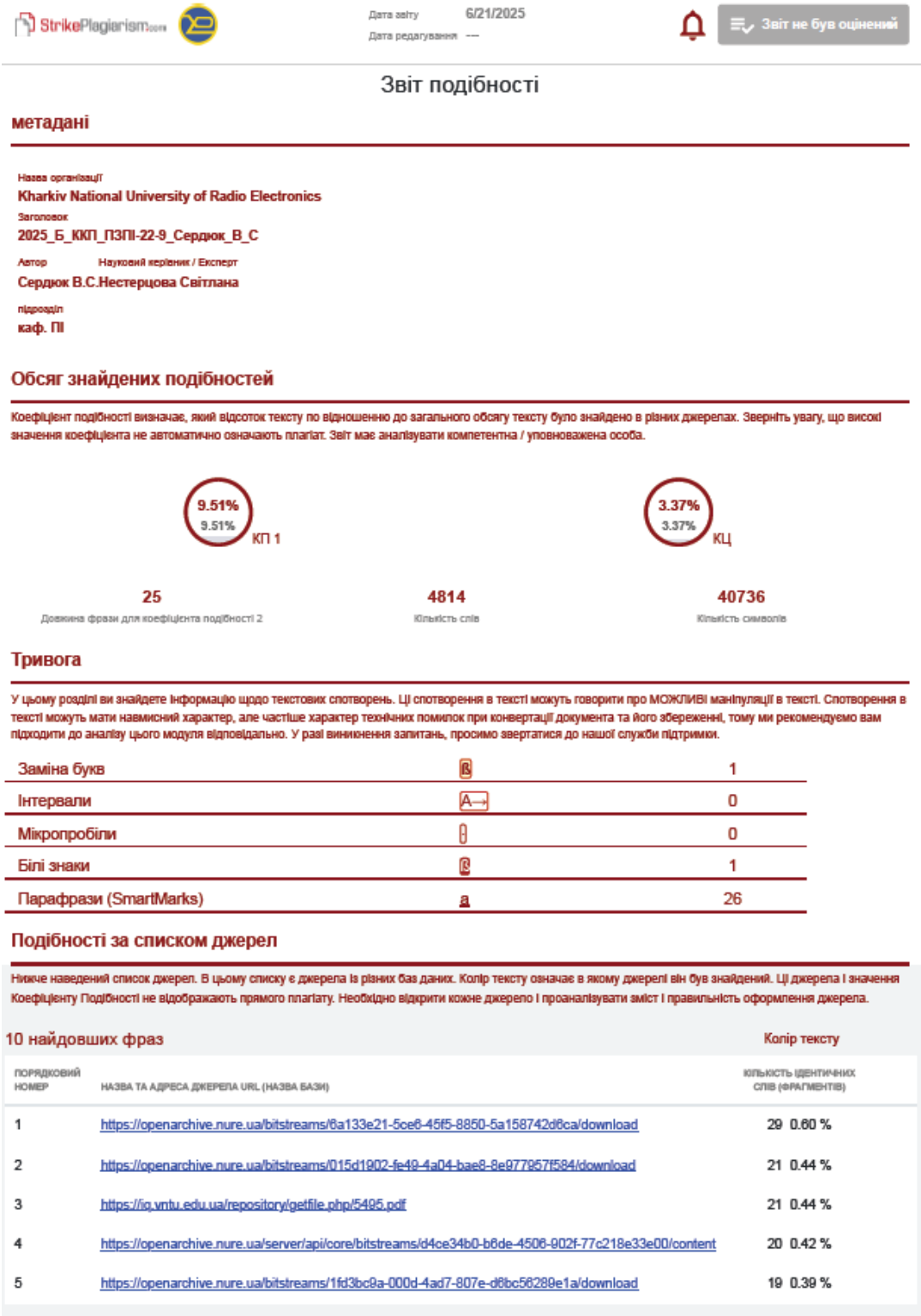
У майбутньому для покращення програми слід додати панель адміністратора для зручного відстеження прогресу користувачів та зручного додавання нових тестів.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Barklund M. React in depth. Manning, 2024. 432 p.
2. Brown E. Web development with node and express. 2019. 340 p.
3. Busuu. URL: <https://www.busuu.com/> (дата звернення: 08.05.2025).
4. Casciaro M., Mammino L. Node.js design patterns. 3rd ed. 2020. 660 p.
5. Martin R. C. Clean code. 2009. 464 p.
6. Memrise. URL: <https://www.memrise.com/> (дата звернення: 07.05.2025).
7. Node.js – Run JavaScript Everywhere. URL: <https://nodejs.org/> (дата звернення: 28.04.2025).
8. Patterns. URL: <https://www.patterns.dev/> (дата звернення: 23.04.2025).
9. React. URL: <https://react.dev/> (дата звернення: 27.04.2025).
10. Refactoring and design patterns. URL: <https://refactoring.guru/> (дата звернення: 06.05.2025).
11. Архів роботи на GitHub. URL:
https://github.com/NureSerdiukViktoriiia/2025_B_KKP_PZPI-22-9_Serdiuk_V_S (дата звернення: 14.06.2025)

ДОДАТОК А

Звіт результатів перевірки на унікальність тексту в базі ХНУРЕ



6	https://www.tempmail.us.com/uk/registration/%D0%97%D0%B0%D0%BF%D0%BE%D0%B1%D1%96%D0%B3%D0%B0%D0%BD%D0%BD%D1%8F-%D0%BF%D0%BE%D0%B2%D1%82%D0%BE%D1%80%D0%BD%D0%B8%D0%BC-%D1%80%D0%B5%D1%94%D1%81%D1%82%D1%80%D0%B0%D1%86%D1%96%D1%8F%D0%BC-%D1%83-mongodb-%D0%B7%D0%BD-%D0%B4%D0%BE%D0%BF%D0%BE%D0%BC%D0%BE%D0%B3%D0%BE%D1%8E-mongoose	19 0.39 %
7	https://openarchive.nure.ua/bitstreams/6a133e21-5ce6-45f5-8850-5a158742d6ca/download	18 0.37 %
8	https://openarchive.nure.ua/bitstreams/8db0c8b6-e850-4696-ba21-a71a64b010cd/download	17 0.35 %
9	https://openarchive.nure.ua/bitstreams/7f91b0b8-31ed-4823-9a92-9261b3f25940/download	16 0.33 %
10	2025 ІПЗ-42 Костанюк А.В. ПОЯСНЮВАЛЬНА ЗАПИСКА 6/10/2025 Volyn Professional College, National University of Food Technologies (Volyn Professional College, National University of Food Technologies)	15 0.31 %
з бази даних RefBooks (0.00 %)		
ПОРЯДКОВИЙ НОМЕР	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
з домашньої бази даних (0.00 %)		
ПОРЯДКОВИЙ НОМЕР	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
з програми обміну базами даних (1.14 %)		
ПОРЯДКОВИЙ НОМЕР	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	2025 ІПЗ-42 Костанюк А.В. ПОЯСНЮВАЛЬНА ЗАПИСКА 6/10/2025 Volyn Professional College, National University of Food Technologies (Volyn Professional College, National University of Food Technologies)	26 (2) 0.54 %
2	Prompt_Engineering_Interactive_Systems_with_Large_Language_Models__An_End_User_Experience_Report.pdf 12/29/2023 University of Southern Denmark (SDU backlog)	13 (2) 0.27 %
3	122_Zubenko_Maksym_Ivanovych_it2025 5/25/2025 National University of Food Technologies (Кафедра інформаційних технологій, штучного інтелекту і кібербезпеки)	6 (1) 0.12 %
4	Веб-система с формування метаданих аудіофайлів 3/15/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	5 (1) 0.10 %
5	Приходько_Данііл_409_ІПЗ_2025 6/13/2025 Ukrainian national aviation university (Фаховий коледж інженерії, управління та землевпорядкування Національного авіаційного університету)	5 (1) 0.10 %
з Інтернету (8.37 %)		
ПОРЯДКОВИЙ НОМЕР	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	https://forum.freecodecamp.org/t/how-to-set-an-active-session-after-registration/667127	54 (7) 1.12 %
2	https://iq.vntu.edu.ua/repository/getfile.php/5495.pdf	54 (6) 1.12 %

Рисунок А.2 – Друга сторінка звіту

3	https://openarchive.nure.ua/bitstreams/6a133e21-5ce6-45f5-8850-5a158742d6ca/download	47 (2) 0.98 %
4	https://openarchive.nure.ua/bitstreams/015d1902-fe49-4a04-bae8-8e977957f584/download	36 (2) 0.75 %
5	https://openarchive.nure.ua/bitstreams/1fd3bc9a-000d-4ad7-807e-d8bc56289e1a/download	30 (2) 0.62 %
6	https://openarchive.nure.ua/bitstreams/b460c359-3771-4d32-b4ff-ce27e10f5835/download	29 (4) 0.60 %
7	https://openarchive.nure.ua/bitstreams/8db0c8b6-e650-4696-ba21-a71a64b010cd/download	23 (2) 0.48 %
8	https://openarchive.nure.ua/server/api/core/bitstreams/d4ce34b0-b6de-4506-902f-77c218e33e00/content	20 (1) 0.42 %
9	https://www.tempmail.us.com/uk/registration/%D0%97%D0%B0%D0%BF%D0%BE%D0%B1%D1%86%D0%B3%D0%B0%D0%BD%D0%BD%D1%8F-%D0%BF%D0%BE%D0%82%D1%82%D0%BE%D1%80%D0%BD%D0%B8%D0%BC-%D1%80%D0%B5%D1%94%D1%81%D1%82%D1%80%D0%B0%D1%86%D1%96%D1%8F%D0%BC-%D1%83-mongodb-%D0%B7%D0%BD-%D0%B4%D0%BE%D0%BF%D0%BE%D0%BC%D0%BE%D0%B3%D0%BE%D1%8E-mongoose	19 (1) 0.39 %
10	https://openarchive.nure.ua/bitstreams/7f91b0b8-31ed-4823-8a92-9261b3f25940/download	16 (1) 0.33 %
11	https://openarchive.nure.ua/bitstreams/e84c9972-be3f-41bb-a499-62f315a08e4c/download	12 (1) 0.25 %
12	https://ua-referat.com/uploaded/metodichni-rekomendaciyi-do-vikonannya-kursovoyi-roboti-dlya-s/index10.html	11 (1) 0.23 %
13	https://openarchive.nure.ua/server/api/core/bitstreams/ad9f1307-d66e-4247-a961-9ce9659ea364/content	10 (1) 0.21 %
14	https://openarchive.nure.ua/bitstreams/dcb885f6-768f-4683-a4bb-5cb937468862/download	10 (1) 0.21 %
15	https://openarchive.nure.ua/bitstreams/a319d575-b7c4-476e-b0fb-623e4d0a5be3/download	9 (1) 0.19 %
16	https://openarchive.nure.ua/bitstreams/f22ea509-4345-4435-a69d-f8c07e54e051/download	7 (1) 0.15 %
17	https://openarchive.nure.ua/bitstreams/b761f519-b4da-4c4e-9097-3c7182d86a99/download	6 (1) 0.12 %
18	https://openarchive.nure.ua/bitstreams/844269c9-b132-4fa8-91d7-3d0bfaee5e4f/download	5 (1) 0.10 %
19	https://openarchive.nure.ua/server/api/core/bitstreams/7e10212e-38c0-46be-a84e-af136ce5c3ff/content	5 (1) 0.10 %

Список прийнятих фрагментів (немає прийнятих фрагментів)

ПОРЯДКОВИЙ НОМЕР

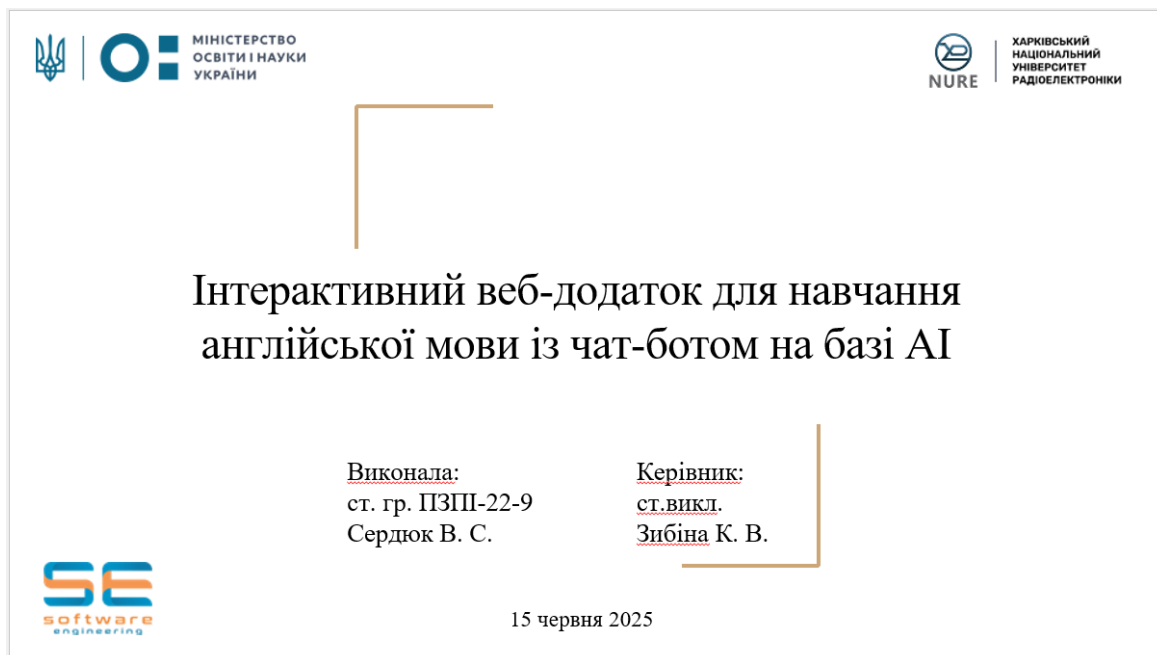
ЗМІСТ

КІЛЬКІСТЬ ОДНАКОВИХ СЛІВ (ФРАГМЕНТІВ)

Рисунок А.3 – Третя сторінка звіту

ДОДАТОК Б

Слайди презентації



МІНІСТЕРСТВО
ОСВІТИ І НАУКИ
УКРАЇНИ

NURE
ХАРКІВСЬКИЙ
НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Інтерактивний веб-додаток для навчання
англійської мови із чат-ботом на базі AI

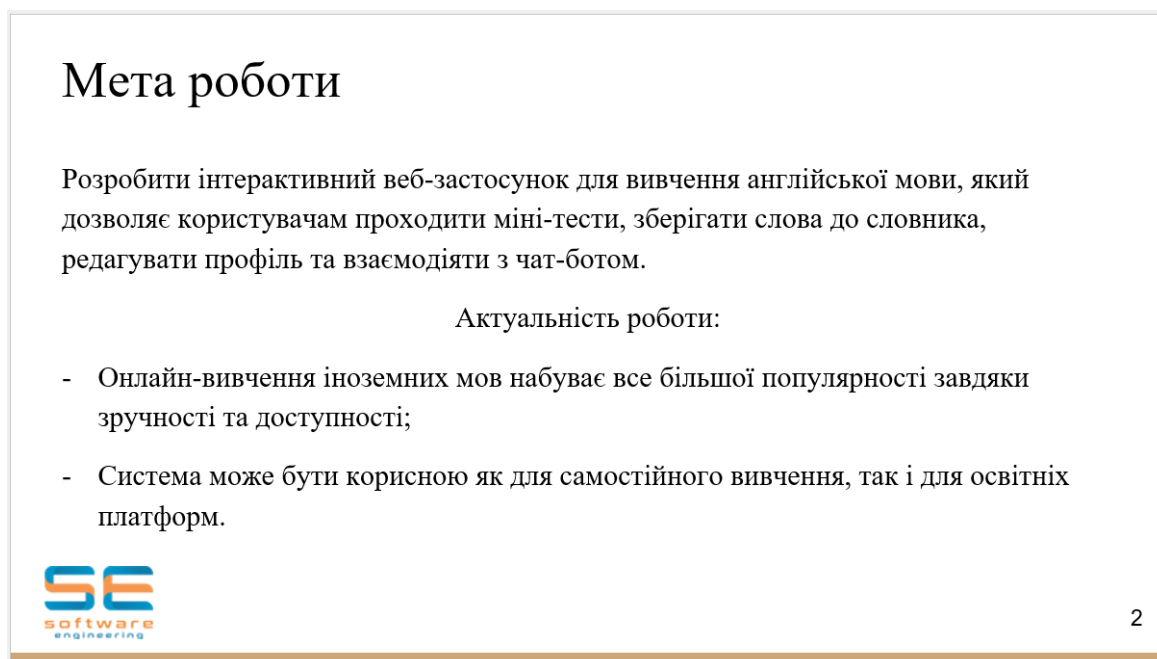
Виконала:
ст. гр. ПЗП-22-9
Сердюк В. С.

Керівник:
ст. викл.
Зибіна К. В.

SE
software
engineering

15 червня 2025

Рисунок Б.1 – Перший слайд



Мета роботи

Розробити інтерактивний веб-застосунок для вивчення англійської мови, який дозволяє користувачам проходити міні-тести, зберігати слова до словника, редагувати профіль та взаємодіяти з чат-ботом.

Актуальність роботи:

- Онлайн-вивчення іноземних мов набуває все більшої популярності завдяки зручності та доступності;
- Система може бути корисною як для самостійного вивчення, так і для освітніх платформ.

SE
software
engineering

2

Рисунок Б.2 – Другий слайд

Аналіз існуючих рішень



Переваги:

- Структуровані уроки за рівнями;
- Короткі відео для розвитку слухання;
- Детальні пояснення граматики після завдань;
- Можливість перегляду вивченого матеріалу;
- Різноманітні тематичні навчальні курси.

Недоліки:

- Обмежений доступ до курсів середнього та просунутого рівня без платної підписки.



Переваги:

- Запам'ятовування слів через інтерактивні картки;
- Широкий вибір курсів на різні теми та рівні;
- Відео з носіями мови.

Недоліки:

- Відсутність детальних правил граматики.



3

Рисунок Б.3 – Третій слайд

Постановка задачі та опис системи

Розробити інтерактивний веб-застосунок для вивчення англійської мови з реалізацією таких функцій: головна сторінка з описом, сторінка профілю користувача, словник для вивчення лексики, проходження міні-тестів, а також можливість реєстрації та входу до системи.

Очікувані результати:

- Інтуїтивно зрозумілий інтерфейс для реєстрації, авторизації та редагування профілю;
- Можливість проходження міні-тестів з обмеженням у часі та кількості запитань;
- Словник для додавання, збереження та повторення нових слів;
- Надійна система захисту персональних даних та безпечна обробка паролів за допомогою хешування.



4

Рисунок Б.4 – Четвертий слайд

Вибір технологій розробки

Фронтенд:



Бекенд:



База даних:

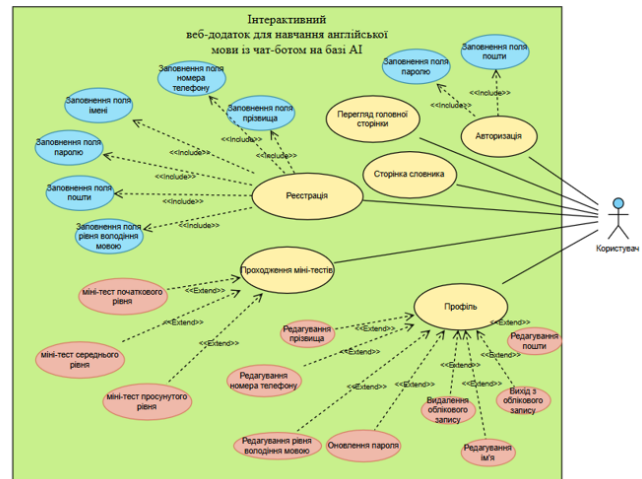


5

Рисунок Б.5 – П'ятий слайд

Архітектура створеного програмного забезпечення

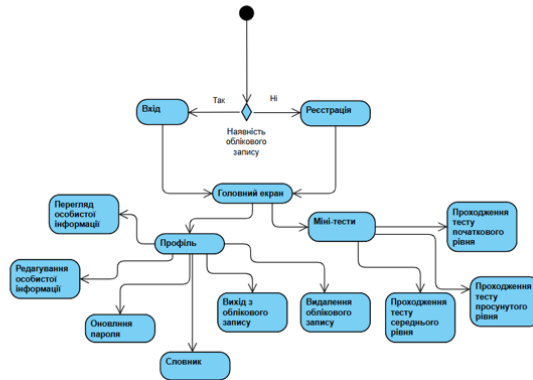
Користувач має можливість зареєструватися, увійти до системи, переглядати та редагувати свій профіль, оновити пароль, видалити обліковий запис, а також взаємодіяти зі сторінкою словника та проходити міні-тести.



6

Рисунок Б.6 – Шостий слайд

Архітектура створеного програмного забезпечення (продовження)



Діаграма діяльності ілюструє послідовність дій користувача в застосунку

Рисунок Б.7 – Сьомий слайд

Дизайн системи

Користувач взаємодіє із застосунком через веб-браузер, який передає запити до веб-сервера за допомогою протоколу HTTP.

У межах веб-сервера визначено два основні компоненти: рівень презентації, який відповідає за відображення інтерфейсу користувача, та рівень доступу до бази даних, який обробляє запити, пов'язані з роботою з даними. Цей рівень взаємодіє з сервером бази даних, де за допомогою локального файлового доступу працює база даних SQLite.

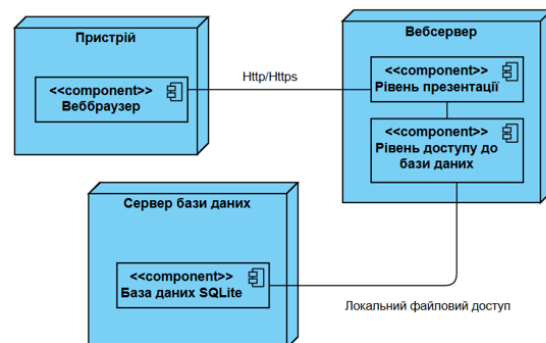


Рисунок Б.8 – Восьмий слайд

Приклад реалізації

```
router.post('/change-password', authenticate, async (req, res) => {
  try {
    const userId = req.user.id;
    const { oldPassword, newPassword } = req.body;

    if (!oldPassword || !newPassword) {
      return res.status(400).json({ message: "Обидва паролі" });
    }

    const user = await User.findById(userId);
    if (!user) {
      return res.status(404).json({ message: "Користувача не знайдено" });
    }

    const isMatch = await bcrypt.compare(oldPassword, user.password);
    if (!isMatch) {
      return res.status(400).json({ message: "Старий пароль невірний" });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(newPassword, salt);
    user.password = hashedPassword;
    await user.save();

    res.json({ message: "Пароль успішно змінено" });
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: "Помилка сервера" });
  }
});
```

Під час зміни пароля система перевіряє старий пароль, хешує новий за допомогою bcrypt і зберігає оновлений пароль у бази даних.

```
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ where: { email } });
    if (!user) {
      return res.status(401).json({ error: "Невірна пошта або пароль" });
    }
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ error: "Невірна пошта або пароль" });
    }
    const token = jwt.sign({ id: user.id }, process.env.SECRET_KEY, {
      expiresIn: "1h",
    });
    res.status(200).json({
      message: "Успішний вхід",
      token,
      user: {
        id: user.id,
        first_name: user.first_name,
        second_name: user.second_name,
        email: user.email,
      },
    });
  } catch (err) {
    console.error("Login error:", err);
    res.status(500).json({ error: "Помилка сервера під час входу" });
  }
});
```

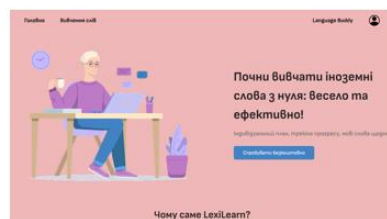
При авторизації система перевіряє, чи існує email у бази даних, порівнює хешований пароль за допомогою bcrypt і, у разі успішної перевірки, генерує токен авторизації, який дозволяє користувачу працювати з системою без повторного входу.



9

Рисунок Б.9 – Дев'ятий слайд

Інтерфейс користувача




10

Рисунок Б.10 – Десятий слайд

Підсумки

Застосунок для навчання англійської мови успішно реалізований і забезпечує основні функції: реєстрацію, авторизацію, роботу з профілем, словником і міні-тестами. Система є зручною та безпечною і підходить як для самостійного навчання, так і для інтеграції в освітні платформи. У майбутньому планується розширення функціоналу за рахунок додавання адміністративної панелі та статистики прогресу користувачів



Рисунок Б.11 – Одинадцятий слайд

ДОДАТОК В

Специфікація програмного забезпечення

CS330 Software Engineering

Software Requirements Specification (SRS) Template

Items that are intended to stay in as part of your document are in bold ; explanatory comments are in <i>italic</i> text. Plain text is used where you might insert wording about your project.
--

The document in this file is an annotated outline for specifying software requirements, adapted from the IEEE Guide to Software Requirements Specifications (Std 830-1993).

Tailor this to your needs, removing explanatory comments as you go along. Where you decide to omit a section, keep the header, but insert a comment saying why you omit the data.

Software system for teaching foreign words

SmartCore Developers

Miach Kateryna

Viktoriia Serdiuk

Software Requirements Specification

Document

Version: 1.0

Date: 17.06.2025

Зміст

1. ВСТУП

- 1.1 Огляд продукту
- 1.2 Мета
- 1.3 Межі
- 1.4 Посилання
- 1.5 Означення та аббревіатури

2. ЗАГАЛЬНИЙ ОПИС

- 2.1 Перспективи продукту
- 2.2 Функції продукту
- 2.3 Характеристики користувачів
- 2.4 Загальні обмеження
- 2.5 Припущення й залежності

3. КОНКРЕТНІ ВИМОГИ

- 3.1 Вимоги до зовнішніх інтерфейсів
 - 3.1.1 Інтерфейс користувача
 - 3.1.2 Апаратний інтерфейс
 - 3.1.3 Програмний інтерфейс
 - 3.1.4 Комунікаційний протокол
 - 3.1.5 Обмеження пам'яті
 - 3.1.6 Операції
 - 3.1.7 Функції продукту
 - 3.1.8 Припущення й залежності
- 3.2 Властивості програмного продукту
- 3.3 Атрибути програмного продукту
 - 3.3.1 Надійність
 - 3.3.2 Доступність
 - 3.3.3 Безпека
 - 3.3.4 Супроводжуваність
 - 3.3.5 Переносимість
 - 3.3.6 Продуктивність
- 3.4 Вимоги бази даних
- 3.5 Інші вимоги

1 ВСТУП

1.1 Огляд продукту

У межах цього проєкту буде створено інтерактивний веб-застосунок для вивчення англійської мови. Застосунок працюватиме через веб-браузер і буде орієнтований на користувачів, які бажають покращити свій словниковий запас, граматику та навички спілкування. Однією з ключових особливостей системи стане впровадження чат-бота, що базуватиметься на технологіях штучного інтелекту, який зможе вести діалоги з користувачем, ставити запитання, виправляти помилки та мотивувати до навчання. Застосунок матиме низку функціональних сторінок: реєстрацію, вхід, профіль, словник, сторінку міні-тестів, сторінки вправ: вибір правильної відповіді та побудова речення, а також взаємодію з чат-ботом. Передбачається, що користувач зможе проходити короткі тести для самоперевірки, переглядати вивчені слова, а також отримувати зворотний зв'язок від AI. Реалізація продукту планується з використанням такого стека технологій:

- Frontend: HTML, CSS, JavaScript, React;
- Backend: Node.js з Express.js;
- База даних: Sequelize ORM з SQLite.

1.2 Мета

Головна мета – розробити ефективний онлайн-інструмент для самостійного вивчення англійської мови з використанням інноваційних підходів до навчання. Продукт має забезпечити зручність, доступність та гнучкість у навчальному процесі, завдяки можливості навчатися у будь-який час і з будь-якого пристрою. Система буде спрямована на: адаптацію навчального матеріалу до рівня користувача; інтеграцію чат-бота, який допомагатиме у вивченні мови в режимі реального часу, генеруватиме відповіді, ставитиме питання та надаватиме індивідуальні рекомендації.

1.3 Межі

На першому етапі розробки застосунок матиме обмежений функціонал, орієнтований на основні потреби користувача, а саме:

- підтримка лише англійської мови;

- відсутність повноцінної адміністративної панелі для керування контентом;
- база даних SQLite, яка обмежена у продуктивності при високому навантаженні;
- історія проходження тестів не зберігатиметься;
- профіль користувача не міститиме повної статистики про активність;
- використання локального середовища для запуску.

1.4 Посилання

Для розробки проєкту можна скористатися такими офіційними джерелами:

React – бібліотека для створення інтерфейсів користувача (<https://react.dev/>)

Node.js – серверна платформа для JavaScript (<https://nodejs.org/>)

Express.js – фреймворк для створення веб-серверів (<https://expressjs.com/>)

Sequelize ORM – ORM для роботи з базами даних (<https://sequelize.org/>)

SQLite – легка вбудована реляційна база даних (<https://sqlite.org/>)

Google Gemini – платформа штучного інтелекту від Google (<https://gemini.google.com/>)

1.5 Означення та аббревіатури

AI (Artificial Intelligence) – Штучний інтелект

JWT (JSON Web Token) – формат для безпечної передачі токенів авторизації

ORM (Object-Relational Mapping) – технологія для взаємодії з базами даних у вигляді об'єктів

SQLite Легка реляційна база даних, що не потребує окремого сервера

GitHub – Платформа для спільної розробки та зберігання коду

2 ЗАГАЛЬНИЙ ОПИС

2.1 Перспективи продукту

Проектований інтерактивний веб-додаток буде самостійним програмним продуктом, призначеним для кінцевих користувачів, що вивчають англійську мову. Він не є частиною більшої системи, проте його архітектура має передбачати можливість майбутньої інтеграції з іншими освітніми платформами або розширення функціонала.

Продукт буде позиціонуватися на ринку як комплексна екосистема для самостійного навчання, що охоплює всі етапи: від визначення рівня знань до щоденної практики. У майбутньому перспективи продукту включатимуть розробку повноцінних мобільних додатків, розширення бази мов для вивчення та інтеграцію з системами управління навчанням.

2.2 Функції продукту

Система буде надавати користувачам наступний набір основних функцій:

- головна сторінка: виконує роль інформаційного лендингу, що презентує основні можливості та переваги додатку;
- реєстрація користувача: користувачі можуть створювати новий обліковий запис, надаючи свої дані;
- автентифікація: зареєстровані користувачі можуть входити до системи за допомогою email та пароля;
- профіль користувача: доступ до особистого профілю з можливістю редагування даних і рівня складності;
- вибір вправ: сторінка з вибором трьох типів інтерактивних вправ – «вибір правильної відповіді», «побудова речення» та «переклад слів»;
- проходження адаптивних вправ: автоматичний підбір завдань відповідно до рівня користувача;
- інтерактивний ai-чат: можливість вести текстовий діалог з ai-асистентом на вільну або обрану тему;
- ведення персонального словника: можливість додавати нові слова до особистого словника під час виконання вправ.

2.3 Характеристики користувачів

Продукт орієнтований на широку аудиторію користувачів, які прагнуть вивчити або покращити англійську мову. Система розрахована на два основні типи користувачів: нового користувача, який може ознайомитися з головною сторінкою, та зареєстрованого користувача, якому доступний весь функціонал. Програмне забезпечення буде підтримувати учнів з різним рівнем підготовки, від початкового до просунутого, адаптуючи контент відповідно до вказаного ними рівня. Для взаємодії з системою від користувача очікується лише базовий рівень комп'ютерної грамотності, що включає вміння користуватися веб-браузером. Основною мотивацією цільової аудиторії є бажання вивчити мову "з нуля", покращити існуючі навички для кар'єрного росту або підготуватися до подорожей.

2.4 Загальні обмеження

При розробці та функціонуванні системи необхідно враховувати низку загальних обмежень. Перш за все, існують технологічні обмеження, оскільки продукт є веб-додатком і вимагатиме від користувача наявності пристрою з доступом до мережі Інтернет та сучасного веб-браузера. Важливим обмеженням є залежність від зовнішніх сервісів, адже функціонал AI чат-бота буде повністю залежати від стабільності та умов надання сервісу Google Gemini API. Також слід враховувати обмеження обраної бази даних: СУБД SQLite, хоча і є зручною для розробки, може мати обмеження продуктивності при високому навантаженні. Нарешті, на поточному етапі існують мовні обмеження, оскільки система призначена виключно для вивчення англійської мови україномовними користувачами.

2.5 Припущення й залежності

Проектування системи базується на кількох ключових припущеннях. Припускається, що користувач матиме стабільне інтернет-з'єднання, необхідне для коректної роботи всіх мережевих функцій, особливо взаємодії з AI-асистентом. Система має низку залежностей, що впливають на її функціонування. Проект критично залежить від стабільності роботи зовнішнього сервісу Google Gemini API. Крім того, він залежить від сторонніх програмних бібліотек та пакетів (npm-

пакетів), таких як React, Express та Sequelize, майбутні оновлення яких можуть вимагати адаптації кодової бази. Доступність додатку для кінцевих користувачів буде залежати від надійності та стабільності роботи обраного хостинг-провайдера.

3. КОНКРЕТНІ ВИМОГИ

3.1 Вимоги до зовнішніх інтерфейсів

3.1.1 Інтерфейс користувача

Інтерфейс користувача повинен бути розроблений з урахуванням принципів UX/UI, забезпечуючи легкість навігації, приємний дизайн і зручність користування. Він має бути адаптивним і коректно відображатися на різних типах пристроїв та в різних браузерах. Основні компоненти інтерфейсу включають:

- Форми реєстрації та авторизації із валідацією введених даних та повідомленнями про помилки;
- Особистий кабінет користувача з можливістю перегляду та редагування персональної інформації та зміни пароля;
- Сторінки словника для додавання, видалення та повторення слів;
- Сторінка з міні-тестами із таймером, підрахунком балів та відображенням результатів;
- Сторінки вправ, що включають вибір правильної відповіді та побудову речення, що допомагають закріпити граматичні та лексичні навички;
- Чат-бот на базі AI для інтерактивної взаємодії та навчання користувача.

3.1.2 Апаратний інтерфейс

Програмний продукт буде працювати на стандартних пристроях з операційними системами Windows, macOS, Linux. Не передбачається використання спеціального обладнання. Всі необхідні обчислювальні ресурси для роботи обробляються на сервері, клієнтською частиною є веб-браузер. Мінімальні апаратні вимоги для клієнтського пристрою:

- Процесор з тактовою частотою не менше 1.5 ГГц;
- Оперативна пам'ять – не менше 2 ГБ;

- Доступ до стабільного інтернет-з'єднання для взаємодії з сервером.

3.1.3 Програмний інтерфейс

Програмне забезпечення буде реалізоване за принципом клієнт-серверної архітектури. Клієнтська частина створюватиметься з використанням бібліотеки React, що дає змогу розробити динамічний і зручний інтерфейс користувача. Серверна частина працюватиме на платформі Node.js із фреймворком Express.js, який дозволяє ефективно обробляти HTTP-запити та легко масштабувати систему. Обмін даними між клієнтом і сервером відбуватиметься через REST API, що забезпечить чітку структуру запитів і стабільну взаємодію компонентів. Формат передавання даних - JSON, який є загальноприйнятим і зручним для веб-розробки. Для роботи з базою даних буде використано ORM Sequelize, який спрощує доступ до даних завдяки об'єктно-орієнтованому підходу. Також планується використання бази даних SQLite.

3.1.4 Комунікаційний протокол

Програмне забезпечення буде реалізовано за принципом взаємодії клієнтської та серверної частин через протокол HTTP або HTTPS. Для обміну даними між клієнтом і сервером використовуватиметься REST API з передачею інформації у форматі JSON. Планується впровадження механізму автентифікації користувачів за допомогою токенів (JWT), що дозволить забезпечити безпечний доступ до системи без повторного входу. З метою захисту персональних даних паролі користувачів будуть хешуватися за допомогою алгоритму bcrypt перед збереженням у базі даних. У фінальній версії, при розгортанні в мережі Інтернет, система використовуватиме безпечний протокол HTTPS для шифрування всього трафіку між клієнтом і сервером.

3.1.5 Обмеження пам'яті

Програма орієнтована на роботу на сучасних пристроях з мінімальними апаратними вимогами, що включають не менше 2 ГБ оперативної пам'яті на клієнтському пристрої. Серверна частина буде розгорнута з урахуванням обмежень доступної пам'яті на хостингу або сервері, з можливістю масштабування при збільшенні кількості користувачів. Для бази даних на

початковому етапі планується використання SQLite, що має невеликі вимоги до ресурсів, з подальшим переходом на більш потужні СУБД, наприклад, MySQL чи PostgreSQL.

3.1.6 Операції

У процесі взаємодії з системою користувач зможе здійснювати такі дії:

- Зареєструвати новий обліковий запис, ввівши необхідні дані;
- Авторизуватися в системі з використанням електронної пошти та пароля;
- Переглядати, редагувати свій профіль, змінювати пароль або видаляти обліковий запис;
- Переглядати сторінку словника з можливістю перекладу та вивчення слів;
- Проходити інтерактивні вправи, такі як: вибір правильної відповіді, переклад слова, побудова речення;
- Отримувати результати виконання вправ;
- Користуватися функціями штучного інтелекту для покращення навчання.

3.1.7 Функції продукту

Програмний продукт повинен забезпечити реалізацію таких основних функцій:

- Реєстрація нових користувачів та вхід до особистого кабінету;
- Керування профілем: редагування особистих даних, зміна пароля, видалення облікового запису;
- Доступ до модуля словника для перегляду, вивчення та перекладу лексики;
- Проходження інтерактивних вправ, зокрема: вибір правильної відповіді, переклад слова, побудова речень;
- Обробка відповідей користувача та надання зворотного зв'язку щодо правильності;
- Взаємодія між клієнтською та серверною частинами через REST API з обміном даними у форматі JSON;
- Інтеграція з AI-сервісом (Google Gemini) для адаптації навчального контенту та підвищення ефективності навчання.

3.1.8 Припущення й залежності

- Користувач має доступ до сучасного веб-браузера, що підтримує JavaScript та актуальні вебтехнології;
- Пристрій користувача відповідає мінімальним технічним вимогам, необхідним для стабільної роботи вебзастосунку;
- Забезпечено стабільне інтернет-з'єднання для коректної взаємодії клієнтської частини з сервером у режимі реального часу;
- На етапі розробки база даних реалізується на SQLite;
- Програмне забезпечення включає інтеграцію з AI-сервісами для реалізації інтелектуальних функцій: адаптації матеріалу до рівня користувача, надання розумних підказок тощо;
- Усі взаємодії з системою (реєстрація, вхід, виконання вправ) реалізуються через REST API з використанням формату JSON та механізмів автентифікації.

3.2 Властивості програмного продукту

Програмний продукт, що розробляється, повинен володіти низкою ключових властивостей, які визначають його якість та відповідність очікуванням користувачів.

По-перше, система повинна бути гнучкою та адаптивною. Це означає, що навчальний контент, зокрема вправи та складність діалогів з AI-асистентом, має динамічно підлаштовуватися під рівень знань, вказаний користувачем.

По-друге, продукт має бути інтерактивним та залучаючим. Взаємодія з системою не повинна зводитися до пасивного споживання інформації. Користувач повинен активно брати участь у процесі: виконувати різноманітні завдання, вести діалог, самостійно формулювати відповіді та отримувати миттєвий зворотний зв'язок.

По-третє, система повинна бути надійною та стабільною. Це передбачає коректну обробку помилок, таких як втрата інтернет-з'єднання або недоступність зовнішніх сервісів, та надання користувачеві зрозумілих повідомлень про стан системи.

По-четверте, продукт має бути розширюваним. Архітектура повинна дозволяти в майбутньому легко додавати нові типи вправ, нові теми для обговорення в чаті та підключати інші мови для вивчення без необхідності кардинальної перебудови всієї системи.

3.3 Атрибути програмного продукту

3.3.1 Надійність

Система повинна залишатися працездатною при виникненні непередбачуваних ситуацій. Вона має коректно обробляти помилки, що надходять від зовнішнього Google Gemini API, і відображати користувачеві відповідне повідомлення, не перериваючи роботу всього додатку. У випадку помилок при збереженні даних у базу, система повинна інформувати про це користувача, а не завершуватися з критичною помилкою.

3.3.2 Доступність

Веб-додаток повинен бути доступним для користувачів 24 години на добу, 7 днів на тиждень, за винятком планових технічних робіт. Очікуваний показник доступності для продуктивного середовища має становити не менше 99.5%.

3.3.3 Безпека

Безпека є критично важливим атрибутом. Система повинна забезпечувати:

- конфіденційність даних користувача: всі дані, що передаються між клієнтом та сервером, повинні шифруватися за допомогою протоколу HTTPS;
- безпечне зберігання паролів: паролі користувачів не повинні зберігатися у відкритому вигляді, вони мають хешуватися за допомогою надійного алгоритму (наприклад, bcrypt);
- автентифікація та авторизація: доступ до персональних даних (словник, профіль) та захищених функцій (проходження вправ, чат) має надаватися лише

автентифікованим користувачам, для цього використовується механізм на основі JWT (JSON Web Tokens);

3.3.4 Супроводжуваність

Кодова база проєкту має бути структурованою та добре документованою. Архітектурне рішення з чітким розділенням на клієнтську та серверну частини, а також модульна структура на обох сторонах (компоненти в React, маршрутизатори в Express) спрощують внесення змін, виправлення помилок та додавання нового функціонала в майбутньому.

3.3.5 Переносимість

Серверна частина, розроблена на Node.js, є кросплатформною і може бути розгорнута на будь-якій операційній системі (Windows, macOS, Linux). Клієнтська частина є веб-додатком і вимагає лише наявності сучасного веб-браузера, що робить її незалежною від платформи користувача.

3.3.6 Продуктивність

Система повинна забезпечувати комфортний час відгуку. Час завантаження основних сторінок не повинен перевищувати 3 секунди на середньостатистичному інтернет-з'єднанні. Відповідь сервера на запити клієнта (крім запитів до AI) не повинна перевищувати 500 мілісекунд. Час відповіді від AI-чату залежить від зовнішнього сервісу, але очікується в межах 2-5 секунд.

3.4 Вимоги бази даних

Перш за все, система повинна гарантувати цілісність даних. Це досягається шляхом використання механізмів первинних (Primary Key) та зовнішніх (Foreign Key) ключів у рамках реляційної моделі.

Другою вимогою є структуроване зберігання. База даних повинна мати чітко визначену схему, що складається з основних таблиць, які відповідають ключовим сутностям предметної області. Поля цих таблиць повинні використовувати відповідні типи даних, такі як INTEGER, VARCHAR, TEXT та DATETIME, для забезпечення коректності та типізації збереженої інформації, що є важливим для подальшої роботи ORM Sequelize.

Третя вимога – ефективність виконання запитів. Структура таблиць та їх індексація, зокрема автоматичне індексування первинних та унікальних ключів (наприклад, id та email у таблиці Users), повинні забезпечувати швидке виконання основних операцій. До таких операцій належать: вибірка користувача за ідентифікатором, пошук питань за рівнем та типом, отримання персонального словника, а також операції вставки та оновлення даних.

Четвертою вимогою є масштабованість. Хоча на початковому етапі для спрощення розробки та розгортання використовується файлова СУБД SQLite, спроектована схема даних є універсальною. Завдяки використанню ORM Sequelize, система спроектована таким чином, щоб у майбутньому можна було легко мігрувати на більш потужні клієнт-серверні СУБД. Вимоги до бази даних мають охоплювати всю систему, а не тільки такі як PostgreSQL або MySQL, без значних змін у бізнес-логіці додатку.

П'ятою вимогою є забезпечення можливості резервного копіювання. Повинна бути передбачена процедура для створення регулярних резервних копій файлу бази даних, що є критично важливим для запобігання втраті даних у разі апаратних збоїв чи інших непередбачуваних ситуацій.

3.5 Інші вимоги

Усі компоненти проєкту мають відповідати встановленим технічним вимогам. Програмний код повинен бути написаний згідно зі стандартами стилю для відповідної мови, бути структурованим, читабельним і містити коментарі в складних місцях для полегшення підтримки. Розробка має вестися з використанням системи контролю версій Git, а також передбачати наявність віддаленого репозиторію для командної роботи, резервного копіювання та

відстеження змін. Серверна частина повинна реалізовувати логування ключових подій та помилок, що дозволить ефективно моніторити систему й вчасно реагувати на збої. Оскільки проєкт використовує зовнішній сервіс Google Gemini API, необхідно подбати про безпечне зберігання API-ключа через файл `.env`, не допускаючи його потрапляння у відкритий клієнтський код.